

Competitive Programming Notebook

August 6, 2026

Contents

1	Miscellaneous	1	4.2.1	SieveOfEratosthenes	13
1.1	Template	1	4.2.2	SmallestPrimeFactor	13
1.2	fastIO	1	4.2.3	DivisorsGeneration	13
1.3	grid	1	4.2.4	PrimeFactorization	13
1.4	random	1	4.3	Modular Arithmetic	13
1.5	coordinateCompression	1	4.3.1	CRT	13
1.6	grayCode	1	5	Combinatorics	13
1.7	customHashHashTable	1	6	Math	13
2	Data Structures (DS)	2	6.1	Modular Arithmetic	13
2.1	Segment Tree	2	6.1.1	ModularArithmetic	13
2.1.1	SegmentTree	2	6.1.2	ModularIntClass	14
2.1.2	LazySegmentTree	2	6.2	Multiplicative Functions	14
2.1.3	PersistentSegmentTree	2	6.2.1	EulerTotientPhi	14
2.1.4	PersistentSegTreeVector	3	6.3	Matrices	14
2.1.5	SegmentTreeBeats	3	6.3.1	MatrixExponentiation	14
2.2	Binary Indexed Tree (BIT)	4	6.3.2	FastMatrixPower	15
2.2.1	FenwickTree	4	6.4	Linear Algebra	15
2.2.2	FenwickTree2D	4	6.4.1	LinearXorBasis	15
2.2.3	Offline2DFenwickTree	5	6.5	Convolution	15
2.3	Sparse Table	5	6.5.1	FastFourierTransform	15
2.3.1	SparseTable	5	6.5.2	NumberTheoreticTransform	15
2.3.2	SparseTable2D	5	7	Strings	15
2.3.3	SquareSparseTable	5	7.1	String Matching	15
2.4	Trie	6	7.1.1	KMP_PrefixFunction	15
2.4.1	StringTrie	6	7.1.2	RabinKarp	16
2.4.2	BinaryTrie	6	7.1.3	ZFunction	16
2.5	Disjoint Set Union (DSU)	6	7.2	Hashing	16
2.5.1	DSU	6	7.2.1	DoubleStringHashing	16
2.5.2	DSURollback	6	7.3	Aho Corasick	16
2.6	Square Root Decomposition	7	7.3.1	AhoCorasickAutomaton	16
2.6.1	MoAlgorithm	7	7.4	Suffix Structures	17
2.6.2	MoOnSubtrees	7	7.4.1	SuffixArrayLCP	17
2.6.3	MoOnPaths	7	7.4.2	SuffixAutomaton	17
2.7	Balanced Binary Search Tree (BBST)	8	7.5	Palindromes	17
2.7.1	Treap	8	7.5.1	Manacher	17
2.7.2	ImplicitTreap	9	8	Dynamic Programming (DP)	17
2.8	Policy Based Data Structures	10	8.1	DP Optimizations	17
2.8.1	OrderedSet	10	8.1.1	DivideAndConquerDP	17
2.9	Monotonic Data Structures	10	8.1.2	ConvexHullTrick	18
2.9.1	TwoStackQueue	10	8.1.3	ConvexHullTrickRollback	18
3	Graph Theory	10	8.1.4	LiChaoTree	18
3.1	Graph Traversal	10	8.1.5	PersistentLiChaoTree	19
3.1.1	TopoSort	10	8.2	Bitmask DP	19
3.2	Shortest Paths	10	8.2.1	SOS_DP	19
3.2.1	Dijkstra	10	9	Trees	19
3.2.2	FloydWarshall	10	9.1	Lowest Common Ancestor (LCA)	19
3.3	Graph Connectivity	11	9.1.1	BinaryLiftingLCA	19
3.3.1	BridgesAndCutEdges	11	9.2	DSU on Tree	19
3.3.2	ArticulationPoints	11	9.2.1	DSUOnTree_Sack	19
3.3.3	SCC_Tarjan	11	9.3	Heavy Light Decomposition (HLD)	20
3.4	Satisfiability	11	9.3.1	HeavyLightDecomposition	20
3.4.1	2SAT	11	9.4	Centroid Decomposition	20
3.5	Eulerian Path	12	9.4.1	CentroidDecomposition	20
3.5.1	EulerianPath	12	10	Game Theory	21
3.6	Flow and Min Cut	12	11	Geometry	21
3.6.1	DinicMaxFlow	12	11.1	Geometry 2D	21
3.6.2	MinCostMaxFlow	12	11.1.1	Point2D	21
3.7	Matching	12	11.1.2	Line2D	21
3.7.1	HopcroftKarpMatching	12	11.1.3	AngleOperations	22
4	Number Theory	13	11.1.4	PolygonArea	22
4.1	Euclidean Algorithm	13	11.1.5	Circle3Points	22
4.1.1	GCD_LCM	13	11.1.6	CircleLineIntersection	22
4.1.2	ExtendedEuclid	13			
4.2	Primes and Divisors	13			

11.1.7	CircleCircleIntersection	22
11.1.8	ConvexHullAndrew	22
11.1.9	RotatingCalipers	22
11.1.10	MinimumEnclosingCircle	22
11.1.11	HalfPlaneIntersection	23
11.2	Geometry 3D	23
11.2.1	Point3D	23

1 Miscellaneous

1.1 Template

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 // #define int long long
6 #define ll long long
7 #define ld long double
8 #define fi first
9 #define se second
10 #define pb push_back
11 #define eb emplace_back
12 #define all(x) x.begin(), x.end()
13 #define rall(x) x.rbegin(), x.rend()
14 #define ones(n) __builtin_popcountll(n)
15 #define msb(n) (63 - __builtin_clzll(n))
16 #define lsb(n) __builtin_ctzll(n)
17
18 const int N = 2e5 + 5, M = 1e3 + 5, LOG = 31;
19 const int inf = 0x3f3f3f3f;
20 const ll llinf = 0x3f3f3f3f3f3f3f3f;
21 const int MOD = 1e9 + 7;
22 const double eps = 1e-9, PI = acos(-1);
23
24 void testCase() {
25 }
26
27 void preCompute() {
28 }
29
30
31 int32_t main() {
32 #ifdef C_Lion
33     freopen("input.txt", "r", stdin);
34     freopen("output.txt", "w", stdout);
35     freopen("errors.txt", "w", stderr);
36 #else
37     //     freopen("input.in", "r", stdin);
38     //     freopen("output.out", "w", stdout);
39 #endif
40
41     ios_base::sync_with_stdio(false), cin.tie(nullptr), cout.tie(
        nullptr);
42     cout << fixed << setprecision(static_cast<int32_t>(-log10(eps)));
43     preCompute();
44
45     int tc = 1;
46     cin >> tc;
47     for (int TC = 1; TC <= tc; TC++) {
48         // cout << "Case #" << TC << ": ";
49         testCase();
50     }
51 }
52
53 /*
54
55
56
57
58 */
```

1.2 fastIO

```
1 const int MOD = 1e9 + 7;
2 const int BUF_SZ = 1 << 15;
3
4 inline namespace Input {
5     char buf[BUF_SZ];
6     int pos;
7     int len;
8     char next_char() {
9         if (pos == len) {
10             pos = 0;
11             len = (int)fread(buf, 1, BUF_SZ, stdin);
12             if (!len) { return EOF; }
13         }
14         return buf[pos++];
15     }
16
17     int read_int() {
18         int x;
19         char ch;
20         int sgn = 1;
21         while (!isdigit(ch = next_char())) {
22             if (ch == '-' ) { sgn *= -1; }
23         }
24         x = ch - '0';
25         while (isdigit(ch = next_char())) { x = x * 10 + (ch - '0'); }
26         return x * sgn;
27     }
28 } // namespace Input
29 inline namespace Output {
30     char buf[BUF_SZ];
31     int pos;
32
33     void flush_out() {
34         fwrite(buf, 1, pos, stdout);
35         pos = 0;
```

```
36     }
37
38     void write_char(char c) {
39         if (pos == BUF_SZ) { flush_out(); }
40         buf[pos++] = c;
41     }
42
43     void write_int(int x) {
44         static char num_buf[100];
45         if (x < 0) {
46             write_char('-');
47             x *= -1;
48         }
49         int len = 0;
50         for (; x >= 10; x /= 10) { num_buf[len++] = (char)('0' + (x %
            10)); }
51         write_char((char)('0' + x));
52         while (len) { write_char(num_buf[--len]); }
53         write_char('\n');
54     }
55
56     // auto-flush output when program exits
57     void init_output() { assert(atexit(flush_out) == 0); }
58 } // namespace Output
```

1.3 grid

```
1 int dx[] = {0, 0, -1, 1, -1, -1, 1, 1};
2 int dy[] = {-1, 1, 0, 0, -1, 1, -1, 1};
3 char di[] = {'L', 'R', 'U', 'D'};
```

1.4 random

```
1 mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
2 template <typename T>
3 T random(T l, T r) {
4     return uniform_int_distribution(l, r)(rng);
5 }
6
7 vector<int> random(int n) {
8     vector<int> v(n);
9     // shuffle 1
10    shuffle(v.begin(), v.end(), rng);
11    // shuffle 2
12    for (int i = 1; i < n; i++)
13        swap(v[i], v[uniform_int_distribution<int>(0, i)(rng)]);
14    return v;
15 }
```

1.5 coordinateCompression

```
1 struct Compress {
2     map<int, int> m;
3     vector<int> undo;
4
5     Compress(vector<int>& v, int base = 0) {
6         m.clear();
7         set<int> s;
8         for (int x : v) {
9             s.insert(x);
10        }
11        undo.resize(s.size() + base);
12        int i = base;
13        for (int x : s) {
14            undo[i] = x;
15            m[x] = i++;
16        }
17        for (int& x : v)
18            x = m[x];
19    }
20 };
```

1.6 grayCode

```
1 vector<int> grayCode(int n) {
2     vector<int> vec = {0};
3     vector<bool> used(1 << n, false);
4     used[0] = true;
5     while (vec.size() < (1 << n)) {
6         int bit = 0;
7         while (used[vec.back() ^ (1 << bit)])
8             bit++;
9         vec.push_back(vec.back() ^ (1 << bit));
10        used[vec.back() ^ (1 << bit)] = true;
11    }
12    return vec;
13 }
```

1.7 customHashHashTable

```
1 #include <ext/pb_ds/assoc_container.hpp>
2 using namespace __gnu_pbds;
3 // gp_hash_table<int, int> table;
4 const int RANDOM = chrono::high_resolution_clock::now().
    time_since_epoch().count();
5 struct chash {
6     int operator()(int x) const { return x ^ RANDOM; }
7 };
8 // gp_hash_table<int, int, chash> table;
```

2 Data Structures (DS)

2.1 Segment Tree

2.1.1 SegmentTree

```
1 struct SEG {
2     ll sum = 0;
3     SEG() {
4     }
```

```

5     SEG(ll x){
6         sum = x;
7     }
8 };
9 struct segTree {
10     vector<SEG> seg;
11     int sz = 1,n;
12     segTree(int nn){
13         n = nn;
14         while (sz < nn)
15             sz *= 2;
16         seg.assign(2 * sz , SEG());
17     }
18
19     SEG merge(SEG seg1, SEG seg2) {
20         SEG ret;
21         ret.sum = seg1.sum + seg2.sum;
22         return ret;
23     }
24
25     void build(vector<int> &v,int x, int lx, int rx) {
26         if (lx == rx) {
27             seg[x] = SEG(v[lx]);
28             return;
29         }
30         int mid = (lx + rx) / 2;
31         int lf = 2 * x + 1, rt = 2 * x + 2;
32
33         build(v,lf, lx, mid);
34         build(v,rt, mid + 1, rx);
35         seg[x] = merge(seg[lf], seg[rt]);
36     }
37     void build(vector<int> &v){
38         build(v,0, 0, n-1);
39     }
40
41     void update(int i, ll val, int x, int lx, int rx) {
42         if (lx == rx) {
43             seg[x] = SEG(val);
44             return;
45         }
46         int mid = (lx + rx) / 2,lf = 2*x+1,rt= 2*x+2;
47         if(i <= mid)
48             update(i, val, lf, lx, mid);
49         else
50             update(i, val, rt, mid + 1, rx);
51         seg[x] = merge(seg[lf] , seg[rt]);
52     }
53
54     void update(int i, ll val) {
55         update(i, val, 0, 0, n-1);
56     }
57
58     SEG query(int l, int r, int x, int lx, int rx) {
59         if (l <= lx && r >= rx)
60             return seg[x];
61         if (l > rx || r < lx)
62             return SEG();
63
64         int mid = (lx + rx) / 2,lf = 2 * x + 1,rt = 2*x+2;
65
66         return merge(query(l, r, lf, lx, mid) , query(l, r, rt, mid +
67             1, rx));
68     }
69
70     SEG query(int l, int r) {
71         return query(l, r, 0, 0, n-1);
72     }
73 };

```

2.1.2 LazySegmentTree

```

1 struct SEG {
2     ll sum = 0, lz = 0;
3
4     SEG() {
5     }
6
7     SEG(ll x) {
8         sum = x;
9     }
10 };
11
12 struct segTree {
13     vector<SEG> seg;
14     int sz = 1, n;
15     ll NO_OP = 0;
16
17     segTree(int nn) {
18         n = nn;
19         while (sz < nn)
20             sz *= 2;
21         seg.assign(2 * sz, SEG());
22     }
23
24     SEG merge(SEG seg1, SEG seg2) {
25         SEG ret;
26         ret.sum = seg1.sum + seg2.sum;
27         return ret;
28     }
29
30     void build(vector<int> &v, int x, int lx, int rx) {
31         if (lx == rx) {
32             seg[x] = SEG(v[lx]);
33             return;
34         }
35         int mid = (lx + rx) / 2;
36         int lf = 2 * x + 1, rt = 2 * x + 2;
37
38         build(v, lf, lx, mid);
39         build(v, rt, mid + 1, rx);
40         seg[x] = merge(seg[lf], seg[rt]);
41     }
42

```

```

43     void build(vector<int> &v) {
44         build(v, 0, 0, n - 1);
45     }
46
47     void push(int x, int lx, int rx) {
48         if (seg[x].lz != NO_OP) {
49             seg[x].sum += seg[x].lz * (rx - lx + 1);
50             int lf = 2 * x + 1, rt = 2 * x + 2;
51             if (lx != rx) {
52                 seg[lf].lz += seg[x].lz;
53                 seg[rt].lz += seg[x].lz;
54             }
55             seg[x].lz = NO_OP;
56         }
57     }
58
59     void update(int l, int r, ll val, int x, int lx, int rx) {
60         push(x, lx, rx);
61         if (l > rx || r < lx)
62             return;
63         if (l <= lx && r >= rx) {
64             seg[x].lz += val;
65             push(x, lx, rx);
66             return;
67         }
68         int mid = (lx + rx) / 2, lf = 2 * x + 1, rt = 2 * x + 2;
69         update(l, r, val, lf, lx, mid);
70         update(l, r, val, rt, mid + 1, rx);
71         seg[x] = merge(seg[lf], seg[rt]);
72     }
73
74     void update(int l, int r, ll val) {
75         update(l, r, val, 0, 0, sz - 1);
76     }
77
78     SEG query(int l, int r, int x, int lx, int rx) {
79         push(x, lx, rx);
80         if (l <= lx && r >= rx)
81             return seg[x];
82         if (l > rx || r < lx)
83             return SEG();
84
85         int mid = (lx + rx) / 2, lf = 2 * x + 1, rt = 2 * x + 2;
86
87         return merge(query(l, r, lf, lx, mid), query(l, r, rt, mid + 1,
88             rx));
89     }
90
91     SEG query(int l, int r) {
92         return query(l, r, 0, 0, sz - 1);
93     }
94 };

```

2.1.3 PersistentSegmentTree

```

1 struct Node {
2     Node *l, *r;
3     ll val = 0;
4
5     Node(int val) : l(NULL), r(NULL), val(val) {
6     }
7
8     Node() : l(NULL), r(NULL) {
9     }
10
11     Node(Node* l, Node* r) : l(l), r(r) {
12         if (l != NULL) val += l->val;
13         if (r != NULL) val += r->val;
14     }
15
16     void addChild() {
17         l = new Node();
18         r = new Node();
19     }
20 };
21
22 struct PST {
23     int n;
24
25     PST(int n) : n(n + 1) {
26     }
27
28     Node merge(Node x, Node y) {
29         Node ret;
30         ret.val = x.val + y.val;
31         return ret;
32     }
33
34     Node* update(Node* v, int i, int val, int lx, int rx) {
35         if (lx == rx) return new Node(v->val + val);
36         int mid = (lx + rx) / 2;
37         if (!v->l) v->addChild();
38         if (i <= mid) return new Node(update(v->l, i, val, lx, mid), v
39             ->r);
40         return new Node(v->l, update(v->r, i, val, mid + 1, rx));
41     }
42
43     Node* update(Node* v, int i, int val) { return update(v, i, val, 0,
44         n - 1); }
45
46     Node query(Node* v, int l, int r, int lx, int rx) {
47         if (l > rx || r < lx) return {};
48         if (l <= lx && r >= rx) return *v;
49         if (!v->l) v->addChild();
50         int mid = (lx + rx) / 2;
51         return merge(query(v->l, l, r, lx, mid), query(v->r, l, r, mid
52             + 1, rx));
53     }
54
55     Node query(Node* v, int l, int r) { return query(v, l, r, 0, n - 1)
56         ; }
57
58     int getKth(Node* a, Node* b, int k, int lx, int rx) {
59         if (lx == rx) return lx;
60     }

```

```

56     if (!a->l) a->addChild();
57     if (!b->l) b->addChild();
58     int rem = b->l->val - a->l->val;
59     int mid = (lx + rx) / 2;
60     if (rem >= k) return getKth(a->l, b->l, k, lx, mid);
61     return getKth(a->r, b->r, k - rem, mid + 1, rx);
62 }
63
64 int getKth(Node* a, Node* b, int k) { return getKth(a, b, k, 0, n - 1); }
65 };

```

2.1.4 PersistentSegTreeVector

```

1 struct Node {
2     Node *l, *r;
3     int val = 0;
4
5     Node(int val): l(NULL), r(NULL), val(val) {}
6     Node(): l(NULL), r(NULL) {}
7     Node(Node *l, Node *r): l(l), r(r) {
8         if (l != NULL) val += l->val;
9         if (r != NULL) val += r->val;
10    }
11
12    void addChild() {
13        l = new Node();
14        r = new Node();
15    }
16 };
17
18 struct PST {
19     int n;
20     PST(int n): n(n + 1) {}
21
22     Node merge(Node x, Node y) {
23         Node ret;
24         ret.val = x.val + y.val;
25         return ret;
26    }
27
28     Node *set(Node *v, int i, int val, int lx, int rx) {
29         if (lx == rx) return new Node(val);
30         int mid = (lx + rx) / 2;
31         if (!v->l) v->addChild();
32         if (i <= mid) return new Node(set(v->l, i, val, lx, mid), v->r);
33         return new Node(v->l, set(v->r, i, val, mid + 1, rx));
34    }
35
36     Node *set(Node *v, int i, int val) { return set(v, i, val, 0, n - 1); }
37
38     // [l, r] r is included
39     Node query(Node *v, int l, int r, int lx, int rx) {
40         if (l > rx || r < lx) return {};
41         if (l <= lx && r >= rx) return *v;
42         if (!v->l) v->addChild();
43         int mid = (lx + rx) / 2;
44         return merge(query(v->l, l, r, lx, mid), query(v->r, l, r, mid + 1, rx));
45    }
46
47     Node query(Node *v, int l, int r) { return query(v, l, r, 0, n - 1); }
48
49     int getKth(Node *a, Node *b, int k, int lx, int rx) {
50         if (lx == rx) return lx;
51         if (!a->l) a->addChild();
52         if (!b->l) b->addChild();
53         int rem = b->l->val - a->l->val;
54         int mid = (lx + rx) / 2;
55         if (rem >= k) return getKth(a->l, b->l, k, lx, mid);
56         return getKth(a->r, b->r, k - rem, mid + 1, rx);
57    }
58
59     int getKth(Node *a, Node *b, int k) { return getKth(a, b, k, 0, n - 1); }
60 };

```

2.1.5 SegmentTreeBeats

```

1 struct Node {
2     int mn;
3     int mnCnt;
4     int secondMn;
5
6     int mx;
7     int mxCnt;
8     int secondMx;
9
10    int sum;
11
12    int lazyAdd;
13    int lazyEqual;
14    bool isLazyEqual;
15
16    Node() {
17        mx = -inf;
18        secondMx = -inf;
19        mxCnt = 0;
20
21        mn = inf;
22        secondMn = inf;
23        mnCnt = 0;
24
25        sum = 0;
26
27        lazyAdd = 0;
28        lazyEqual = 0;
29        isLazyEqual = 0;
30    };
31
32    Node(int x) {

```

```

33        mx = x;
34        secondMx = -inf;
35        mxCnt = 1;
36
37        mn = x;
38        secondMn = inf;
39        mnCnt = 1;
40
41        sum = x;
42
43        lazyAdd = 0;
44        lazyEqual = 0;
45        isLazyEqual = 0;
46    }
47 };
48
49 struct SegTree {
50     int tree_size;
51     vector<Node> seg_data;
52
53     SegTree(int n) {
54         tree_size = 1;
55         while (tree_size < n) tree_size *= 2;
56         seg_data.resize(2 * tree_size, Node());
57     }
58
59     Node merge(Node& lf, Node& ri) {
60         Node ans = Node();
61
62         ans.sum = lf.sum + ri.sum;
63
64         ans.mx = max(lf.mx, ri.mx);
65         ans.secondMx = max(lf.secondMx, ri.secondMx);
66         ans.mxCnt = 0;
67         if (lf.mx == ans.mx) ans.mxCnt += lf.mxCnt;
68         else ans.secondMx = max(ans.secondMx, lf.mx);
69         if (ri.mx == ans.mx) ans.mxCnt += ri.mxCnt;
70         else ans.secondMx = max(ans.secondMx, ri.mx);
71
72         ans.mn = min(lf.mn, ri.mn);
73         ans.secondMn = min(lf.secondMn, ri.secondMn);
74         ans.mnCnt = 0;
75         if (lf.mn == ans.mn) ans.mnCnt += lf.mnCnt;
76         else ans.secondMn = min(ans.secondMn, lf.mn);
77         if (ri.mn == ans.mn) ans.mnCnt += ri.mnCnt;
78         else ans.secondMn = min(ans.secondMn, ri.mn);
79
80         return ans;
81     }
82
83     void init(vector<int>& nums, int ni, int lx, int rx) {
84         if (rx - lx == 1) {
85             if (lx < nums.size())
86                 seg_data[ni] = Node(nums[lx]);
87             return;
88         }
89
90         int mid = lx + (rx - lx) / 2;
91         init(nums, 2 * ni + 1, lx, mid);
92         init(nums, 2 * ni + 2, mid, rx);
93
94         seg_data[ni] = merge(seg_data[2 * ni + 1], seg_data[2 * ni + 2]);
95     }
96
97     void init(vector<int>& nums) {
98         init(nums, 0, 0, tree_size);
99     }
100
101     void doPushEq(int ni, int lx, int rx, int x) {
102         seg_data[ni].mn = seg_data[ni].mx = seg_data[ni].lazyEqual = x;
103         seg_data[ni].isLazyEqual = 1;
104         seg_data[ni].mnCnt = seg_data[ni].mxCnt = rx - lx;
105         seg_data[ni].secondMx = -inf;
106         seg_data[ni].secondMn = inf;
107
108         seg_data[ni].sum = x * (rx - lx);
109         seg_data[ni].lazyAdd = 0;
110     }
111
112     void doPushMinEq(int ni, int lx, int rx, int x) {
113         if (seg_data[ni].mn >= x) doPushEq(ni, lx, rx, x);
114         else if (seg_data[ni].mx > x) {
115             if (seg_data[ni].secondMn == seg_data[ni].mx)
116                 seg_data[ni].secondMn = x;
117             seg_data[ni].sum -= (seg_data[ni].mx - x) * seg_data[ni].mxCnt;
118             seg_data[ni].mx = x;
119         }
120     }
121
122     void doPushMaxEq(int ni, int lx, int rx, int x) {
123         if (seg_data[ni].mx <= x) doPushEq(ni, lx, rx, x);
124         else if (seg_data[ni].mn < x) {
125             if (seg_data[ni].secondMx == seg_data[ni].mn)
126                 seg_data[ni].secondMx = x;
127
128             seg_data[ni].sum += (x - seg_data[ni].mn) * seg_data[ni].mnCnt;
129             seg_data[ni].mn = x;
130         }
131     }
132
133     void doPushSum(int ni, int lx, int rx, int x) {
134         if (seg_data[ni].mn == seg_data[ni].mx)
135             doPushEq(ni, lx, rx, seg_data[ni].mn + x);
136         else {
137             seg_data[ni].mx += x;
138             if (seg_data[ni].secondMx != -inf)
139                 seg_data[ni].secondMx += x;
140
141             seg_data[ni].mn += x;
142             if (seg_data[ni].secondMn != inf)
143                 seg_data[ni].secondMn += x;

```

```

144         seg_data[ni].sum += (rx - lx) * x;
145         seg_data[ni].lazyAdd += x;
146     }
147 }
148
149 void propagete(int ni, int lx, int rx) {
150     if (rx - lx == 1) return;
151
152     int mid = lx + (rx - lx) / 2;
153     if (seg_data[ni].isLazyEqual) {
154         doPushEq(2 * ni + 1, lx, mid, seg_data[ni].lazyEqual);
155         doPushEq(2 * ni + 2, mid, rx, seg_data[ni].lazyEqual);
156         seg_data[ni].lazyEqual = seg_data[ni].isLazyEqual = 0;
157     }
158     else {
159         doPushSum(2 * ni + 1, lx, mid, seg_data[ni].lazyAdd);
160         doPushSum(2 * ni + 2, mid, rx, seg_data[ni].lazyAdd);
161         seg_data[ni].lazyAdd = 0;
162
163         doPushMinEq(2 * ni + 1, lx, mid, seg_data[ni].mx);
164         doPushMinEq(2 * ni + 2, mid, rx, seg_data[ni].mx);
165
166         doPushMaxEq(2 * ni + 1, lx, mid, seg_data[ni].mn);
167         doPushMaxEq(2 * ni + 2, mid, rx, seg_data[ni].mn);
168     }
169 }
170
171 void minimize(int l, int r, int x, int ni, int lx, int rx) {
172     if (rx <= 1 || lx >= r || seg_data[ni].mx <= x)
173         return;
174     if (lx >= 1 && rx <= r && seg_data[ni].secondMx < x) {
175         doPushMinEq(ni, lx, rx, x);
176         return;
177     }
178
179     propagete(ni, lx, rx);
180
181     int mid = lx + (rx - lx) / 2;
182     minimize(l, r, x, 2 * ni + 1, lx, mid);
183     minimize(l, r, x, 2 * ni + 2, mid, rx);
184
185     seg_data[ni] = merge(seg_data[2 * ni + 1], seg_data[2 * ni +
186         2]);
187 }
188
189 void minimize(int l, int r, int x) {
190     minimize(l, r, x, 0, 0, tree_size);
191 }
192
193 void maximize(int l, int r, int x, int ni, int lx, int rx) {
194     if (rx <= 1 || lx >= r || seg_data[ni].mn >= x)
195         return;
196     if (lx >= 1 && rx <= r && seg_data[ni].secondMn > x) {
197         doPushMaxEq(ni, lx, rx, x);
198         return;
199     }
200
201     propagete(ni, lx, rx);
202
203     int mid = lx + (rx - lx) / 2;
204     maximize(l, r, x, 2 * ni + 1, lx, mid);
205     maximize(l, r, x, 2 * ni + 2, mid, rx);
206
207     seg_data[ni] = merge(seg_data[2 * ni + 1], seg_data[2 * ni +
208         2]);
209 }
210
211 void maximize(int l, int r, int x) {
212     maximize(l, r, x, 0, 0, tree_size);
213 }
214
215 void add(int l, int r, int x, int ni, int lx, int rx) {
216     if (rx <= 1 || lx >= r)
217         return;
218     if (lx >= 1 && rx <= r) {
219         doPushSum(ni, lx, rx, x);
220         return;
221     }
222
223     propagete(ni, lx, rx);
224
225     int mid = lx + (rx - lx) / 2;
226     add(l, r, x, 2 * ni + 1, lx, mid);
227     add(l, r, x, 2 * ni + 2, mid, rx);
228
229     seg_data[ni] = merge(seg_data[2 * ni + 1], seg_data[2 * ni +
230         2]);
231 }
232
233 void add(int l, int r, int x) {
234     add(l, r, x, 0, 0, tree_size);
235 }
236
237 void set(int l, int r, int x, int ni, int lx, int rx) {
238     if (rx <= 1 || lx >= r)
239         return;
240     if (lx >= 1 && rx <= r) {
241         doPushEq(ni, lx, rx, x);
242         return;
243     }
244
245     propagete(ni, lx, rx);
246
247     int mid = lx + (rx - lx) / 2;
248     set(l, r, x, 2 * ni + 1, lx, mid);
249     set(l, r, x, 2 * ni + 2, mid, rx);
250
251     seg_data[ni] = merge(seg_data[2 * ni + 1], seg_data[2 * ni +
252         2]);
253 }
254
255 void set(int l, int r, int x) {
256     set(l, r, x, 0, 0, tree_size);

```

```

254 }
255
256 Node get_range(int l, int r, int ni, int lx, int rx) {
257     propagete(ni, lx, rx);
258
259     if (lx >= 1 && rx <= r)
260         return seg_data[ni];
261     if (rx <= 1 || lx >= r)
262         return Node();
263
264     int mid = (lx + rx) / 2;
265
266     Node lf = get_range(l, r, 2 * ni + 1, lx, mid);
267     Node ri = get_range(l, r, 2 * ni + 2, mid, rx);
268     return merge(lf, ri);
269 }
270
271 int get_sum(int l, int r) {
272     return get_range(l, r, 0, 0, tree_size).sum;
273 }
274
275 int get_mx(int l, int r) {
276     return get_range(l, r, 0, 0, tree_size).mx;
277 }
278
279 int get_mn(int l, int r) {
280     return get_range(l, r, 0, 0, tree_size).mn;
281 }
282 };

```

2.2 Binary Indexed Tree (BIT)

2.2.1 FenwickTree

```

1 template<typename T>
2 class FenwickTree {
3 public:
4     vector<T> tree;
5     int n;
6
7     // Default constructor
8     FenwickTree() : n(0) {}
9
10    // Constructor with size
11    FenwickTree(int n) {
12        init(n);
13    }
14
15    void init(int n) {
16        this->n = n;
17        // 1-based indexing needs size n + 1. Padding by 2 is safe.
18        tree.assign(n + 2, 0);
19    }
20
21    void update(int x, T val) {
22        x++; // Convert 0-based to 1-based
23        for (; x <= n; x += x & -x) {
24            tree[x] += val;
25        }
26    }
27
28    void assign(int x, T val) {
29        update(x, val - getRange(x, x));
30    }
31
32    T getPrefix(int x) {
33        x++; // Convert 0-based to 1-based
34        if (x <= 0) return 0;
35        T ret = 0;
36        for (; x; x -= x & -x) {
37            ret += tree[x];
38        }
39        return ret;
40    }
41
42    T getRange(int l, int r) {
43        if (l > r) return 0;
44        return getPrefix(r) - getPrefix(l - 1);
45    }
46
47    // Returns the 0-based index of the first prefix sum >= x
48    int lowerBound(T x) {
49        int pos = 0;
50        // sz > 0 is enough; don't break early if x == 0 to handle 0-
51        // value elements properly
52        for (int sz = (1 << __lg(n)); sz > 0; sz >>= 1) {
53            if (pos + sz <= n && tree[pos + sz] < x) {
54                pos += sz;
55            }
56        }
57        return pos; // pos is the 0-based index (since 1-based would be
58        pos + 1)
59    };

```

2.2.2 FenwickTree2D

```

1 template<typename T>
2 class FenwickTree2D {
3 public:
4     vector<vector<T>> tree;
5     int n, m;
6
7     // Default constructor
8     FenwickTree2D() : n(0), m(0) {}
9
10    // Constructor with dimensions
11    FenwickTree2D(int n, int m) {
12        init(n, m);
13    }
14
15    void init(int n, int m) {
16        this->n = n;

```

```

17     this->m = m;
18     // 1-based indexing needs sizes n + 1 and m + 1. Padding by 2
19     // is safe.
20     tree.assign(n + 2, vector<T>(m + 2, 0));
21 }
22
23 // Point update: Adds 'val' to the cell (x, y)
24 void update(int x, int y, T val) {
25     x++; y++; // Convert 0-based to 1-based
26     for (int i = x; i <= n; i += i & -i) {
27         for (int j = y; j <= m; j += j & -j) {
28             tree[i][j] += val;
29         }
30     }
31 }
32
33 // Point assignment: Sets the cell (x, y) to 'val'
34 void assign(int x, int y, T val) {
35     update(x, y, val - getRange(x, y, x, y));
36 }
37
38 // 2D Prefix sum: Returns the sum of the subgrid from (0, 0) to (x, y)
39 T getPrefix(int x, int y) {
40     x++; y++; // Convert 0-based to 1-based
41     if (x <= 0 || y <= 0) return 0;
42
43     // Clamp to maximum dimensions if queries exceed bounds
44     x = min(x, n);
45     y = min(y, m);
46
47     T ret = 0;
48     for (int i = x; i > 0; i -= i & -i) {
49         for (int j = y; j > 0; j -= j & -j) {
50             ret += tree[i][j];
51         }
52     }
53     return ret;
54 }
55
56 // 2D Range sum: Returns the sum of the subgrid from top-left (x1, y1) to bottom-right (x2, y2)
57 T getRange(int x1, int y1, int x2, int y2) {
58     if (x1 > x2 || y1 > y2) return 0;
59
60     // Inclusion-Exclusion Principle
61     return getPrefix(x2, y2)
62         - getPrefix(x1 - 1, y2)
63         - getPrefix(x2, y1 - 1)
64         + getPrefix(x1 - 1, y1 - 1);
65 }
66 };

```

2.2.3 Offline2DFenwickTree

```

1 template <typename T>
2 struct BIT2D {
3     int n;
4     vector<vector<int>> vals;
5     vector<vector<T>> bit;
6
7     int ind(const vector<int> &v, int x) {
8         return upper_bound(begin(v), end(v), x) - begin(v) - 1;
9     }
10
11     // n: the limit of the first dimension
12     // all update operations you will make
13     BIT2D(int n, vector<array<int, 2>> todo): n(n + 1), vals(n + 1),
14         bit(n + 1) {
15         sort(begin(todo), end(todo), [](auto &a, auto &b) { return a[1] < b[1]; });
16
17         for (int i = 0; i < n; i++) vals[i].push_back(-oo);
18         for (auto [r, c] : todo)
19             for (; r < n; r |= r + 1)
20                 if (vals[r].back() != c) vals[r].push_back(c);
21
22         for (int i = 0; i < n; i++) bit[i].resize(vals[i].size());
23     }
24
25     void add(int r, int c, T val) {
26         for (; r < n; r |= r + 1) {
27             int i = ind(vals[r], c);
28             for (; i < bit[r].size(); i |= i + 1) bit[r][i] += val;
29         }
30     }
31
32     T query(int r, int c) {
33         T ans = 0;
34         for (; r >= 0; r = (r & r + 1) - 1) {
35             int i = ind(vals[r], c);
36             for (; i >= 0; i = (i & i + 1) - 1) ans += bit[r][i];
37         }
38         return ans;
39     }
40
41     T query(int r1, int c1, int r2, int c2) {
42         return query(r2, c2) - query(r2, c1 - 1) - query(r1 - 1, c2) +
43             query(r1 - 1, c1 - 1);
44     }
45
46     void reset() {
47         for(auto &v: bit) for(auto &i: v) i = 0;
48     }
49 };

```

2.3 Sparse Table

2.3.1 SparseTable

```

1 #define sz(aa) (int)aa.size()
2
3 template <class T>
4 struct sparseTable {

```

```

5     vector<vector<T>> jmp;
6
7     void build(const vector<T>& V) {
8         jmp.assign(1, V);
9         for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k) {
10             jmp.emplace_back(sz(V) - pw * 2 + 1);
11             for (int j = 0; j < sz(jmp[k]); ++j) {
12                 jmp[k][j] = max(jmp[k - 1][j], jmp[k - 1][j + pw]);
13             }
14         }
15     }
16
17     T query(int l, int r) {
18         assert(l <= r);
19         int dep = 31 - __builtin_clz(r - l + 1);
20         return max(jmp[dep][l], jmp[dep][r - (1 << dep) + 1]);
21     }
22 };

```

2.3.2 SparseTable2D

```

1 #define sz(aa) (int)aa.size()
2
3 template <class T>
4 struct sparseTable2D {
5     int n, m;
6     // jmp[kr][kc][i][j] stores the max of a 2^kr by 2^kc rectangle
7     // starting at (i, j)
8     vector<vector<vector<vector<T>>>> jmp;
9
10     void build(const vector<vector<T>>& V) {
11         n = sz(V);
12         if (n == 0) return;
13         m = sz(V[0]);
14         if (m == 0) return;
15
16         int log_n = 32 - __builtin_clz(n);
17         int log_m = 32 - __builtin_clz(m);
18
19         jmp.assign(log_n, vector<vector<vector<T>>>(log_m, vector<
20             vector<T>>(n, vector<T>(m))));
21
22         // Base case: 1x1 rectangles (kr = 0, kc = 0)
23         for (int i = 0; i < n; ++i) {
24             for (int j = 0; j < m; ++j) {
25                 jmp[0][0][i][j] = V[i][j];
26             }
27         }
28
29         // Build 1D sparse tables for each row (kr = 0, increasing kc)
30         for (int kc = 1; kc < log_m; ++kc) {
31             for (int i = 0; i < n; ++i) {
32                 for (int j = 0; j + (1 << kc) <= m; ++j) {
33                     jmp[0][kc][i][j] = max(jmp[0][kc - 1][i][j],
34                         jmp[0][kc - 1][i][j + (1 <<
35                             (kc - 1))]);
36                 }
37             }
38         }
39
40         // Build the 2D tables by merging rows (increasing kr, for all kc)
41         for (int kr = 1; kr < log_n; ++kr) {
42             for (int kc = 0; kc < log_m; ++kc) {
43                 for (int i = 0; i + (1 << kr) <= n; ++i) {
44                     for (int j = 0; j + (1 << kc) <= m; ++j) {
45                         jmp[kr][kc][i][j] = max(jmp[kr - 1][kc][i][j],
46                             jmp[kr - 1][kc][i + (1 <<
47                                 (kr - 1))][j]);
48                     }
49                 }
50             }
51         }
52
53         // Queries the maximum in the subgrid from (r1, c1) to (r2, c2)
54         // inclusive
55         T query(int r1, int c1, int r2, int c2) {
56             assert(r1 <= r2 && c1 <= c2);
57             int kr = 31 - __builtin_clz(r2 - r1 + 1);
58             int kc = 31 - __builtin_clz(c2 - c1 + 1);
59
60             // Get the 4 overlapping sub-rectangles
61             T top_left = jmp[kr][kc][r1][c1];
62             T top_right = jmp[kr][kc][r1][c2 - (1 << kc) + 1];
63             T bottom_left = jmp[kr][kc][r2 - (1 << kr) + 1][c1];
64             T bottom_right = jmp[kr][kc][r2 - (1 << kr) + 1][c2 - (1 << kc)
65                 + 1];
66
67             return max({top_left, top_right, bottom_left, bottom_right});
68         }
69     };

```

2.3.3 SquareSparseTable

```

1 #define sz(aa) (int)aa.size()
2
3 template <class T>
4 struct squareSparseTable {
5     int n, m;
6     // jmp[k][i][j] stores the max of a 2^k by 2^k square starting at (
7     // i, j)
8     vector<vector<vector<T>>> jmp;
9
10     void build(const vector<vector<T>>& V) {
11         n = sz(V);
12         if (n == 0) return;
13         m = sz(V[0]);
14         if (m == 0) return;
15
16         // The maximum possible square side length is min(n, m)
17         int max_len = min(n, m);
18         int log_max = 32 - __builtin_clz(max_len);

```

```

18
19 // Only 3 dimensions: log(min(N,M)) x N x M
20 jmp.assign(log_max, vector<vector<T>>(n, vector<T>(m)));
21
22 // Base case: 1x1 squares (k = 0)
23 for (int i = 0; i < n; ++i) {
24     for (int j = 0; j < m; ++j) {
25         jmp[0][i][j] = V[i][j];
26     }
27 }
28
29 // Build larger squares by combining 4 smaller squares
30 for (int k = 1; k < log_max; ++k) {
31     int pw = 1 << (k - 1); // Length of the smaller squares
32     // 2^(k-1)
33     for (int i = 0; i + (pw * 2) <= n; ++i) {
34         for (int j = 0; j + (pw * 2) <= m; ++j) {
35             jmp[k][i][j] = max({
36                 jmp[k - 1][i][j], // Top-Left
37                 jmp[k - 1][i][j + pw], // Top-Right
38                 jmp[k - 1][i + pw][j], // Bottom-Left
39                 jmp[k - 1][i + pw][j + pw] // Bottom-Right
40             });
41         }
42     }
43 }
44
45 // Queries the maximum in a square from (r1, c1) to (r2, c2)
46 T query(int r1, int c1, int r2, int c2) {
47     assert(r1 <= r2 && c1 <= c2);
48     assert((r2 - r1) == (c2 - c1)); // Ensure it is actually a
49     // square
50     int len = r2 - r1 + 1;
51     int k = 31 - __builtin_clz(len);
52     int pw = 1 << k;
53
54     // Overlap four 2^k x 2^k squares to cover the arbitrary S x S
55     // square
56     return max({
57         jmp[k][r1][c1],
58         jmp[k][r1][c2 - pw + 1],
59         jmp[k][r2 - pw + 1][c1],
60         jmp[k][r2 - pw + 1][c2 - pw + 1]
61     });
62 }
63 };

```

2.4 Trie

2.4.1 StringTrie

```

1 struct Trie {
2     vector<vector<int>> trie;
3     vector<int> cnt;
4     // vector<int>leaves;
5     int m, sz;
6
7     int addNode() {
8         trie.emplace_back(m, -1);
9         cnt.emplace_back();
10        // leaves.emplace_back();
11        sz++;
12        return sz - 1;
13    }
14
15    Trie(int m = 26) : m(m), sz(0) {
16        addNode();
17    };
18
19    // insert or remove
20    void insert(string &s, int type = 1) {
21        int cur = 0;
22        for (auto c: s) {
23            if (trie[cur][c - 'a'] == -1)
24                trie[cur][c - 'a'] = addNode();
25            cur = trie[cur][c - 'a'];
26            cnt[cur] += type;
27        }
28        // leaves[cur] += type;
29    }
30
31    int query(string &s) {
32        int cur = 0;
33        for (auto c: s) {
34            if (trie[cur][c - 'a'] == -1)
35                return 0;
36            cur = trie[cur][c - 'a'];
37        }
38        return cnt[cur];
39    }
40 }
41 };

```

2.4.2 BinaryTrie

```

1 struct Trie{
2     vector<vector<int>>trie;
3     vector<int>cnt;
4     // vector<int>leaves;
5     int mxBit,sz;
6
7     int addNode(){
8         trie.emplace_back(2,-1);
9         cnt.emplace_back();
10        // leaves.emplace_back();
11        sz++;
12        return sz - 1;
13    }
14
15    Trie(int mx = 60): mxBit(mx),sz(0){
16        addNode();
17    };

```

```

18
19 // insert or remove
20 void insert(ll x,int type = 1){
21     int cur = 0;
22     cnt[cur] += type;
23     for (int i = mxBit; i >= 0; --i) {
24         int t = (x >> i)&1;
25         if(trie[cur][t] == -1)
26             trie[cur][t] = addNode();
27         cur = trie[cur][t];
28         cnt[cur] += type;
29     }
30     // leaves[cur] += type;
31 }
32
33 ll maxXor(ll x){
34     // no elements in trie
35     int cur = 0;
36     if(!cnt[cur])return -1e9;
37     for (int i = mxBit; i >= 0; --i) {
38         int t = (x >> i)&1^1;
39         if(trie[cur][t] == -1 || !cnt[trie[cur][t]])t ^= 1;
40         cur = trie[cur][t];
41         if(t)x ^= 1ll << i;
42     }
43     return x;
44 }
45 };

```

2.5 Disjoint Set Union (DSU)

2.5.1 DSU

```

1 struct DSU {
2     vector<int> par, sz;
3
4     DSU(int n) : par(n), sz(n, 1) { iota(par.begin(), par.end(), 0); }
5
6     int find(int x) {
7         if(x == par[x])return x;
8         return par[x] = find(par[x]);
9     }
10
11     bool same(int x, int y) { return find(x) == find(y); }
12
13     bool join(int x, int y) {
14         x = find(x);
15         y = find(y);
16         if (x == y) return false;
17         if (sz[x] < sz[y])
18             swap(x, y);
19         sz[x] += sz[y];
20         par[y] = x;
21         return true;
22     }
23
24     int size(int x) { return sz[find(x)]; }
25 };

```

2.5.2 DSURollback

```

1 struct DSURollback {
2     vector<int> par, sz;
3     vector<pair<int, int>> history; // Stores {child_node,
4         // old_size_of_parent}
5
6     DSURollback(int n) : par(n), sz(n, 1) {
7         iota(par.begin(), par.end(), 0);
8     }
9
10    // Path compression is removed to preserve tree structure for
11    // rollbacks
12    int find(int x) {
13        if(x == par[x]) return x;
14        return find(par[x]);
15    }
16
17    bool same(int x, int y) { return find(x) == find(y); }
18
19    bool join(int x, int y) {
20        x = find(x);
21        y = find(y);
22
23        if (x == y) {
24            // Push dummy state to keep the rollback count 1:1 with
25            // join calls
26            history.push_back({-1, -1});
27            return false;
28        }
29
30        if (sz[x] < sz[y]) swap(x, y);
31
32        // Record the node that becomes the child and the parent's
33        // current size
34        history.push_back({y, sz[x]});
35
36        sz[x] += sz[y];
37        par[y] = x;
38        return true;
39    }
40
41    int size(int x) { return sz[find(x)]; }
42
43    // Undoes the last join operation
44    void rollback() {
45        if (history.empty()) return;
46
47        auto [y, old_sz_x] = history.back();
48        history.pop_back();
49
50        if (y != -1) {
51            int x = par[y];
52            sz[x] = old_sz_x;
53            par[y] = y; // Restore the child to be its own parent

```



```

50     }
51 }
52
53 // Returns the current number of operations performed
54 int get_state() {
55     return history.size();
56 }
57
58 // Rolls back to a specific saved state
59 void rollback_to(int state) {
60     while (history.size() > state) {
61         rollback();
62     }
63 }
64 };

```

2.6 Square Root Decomposition

2.6.1 MoAlgorithm

```

1 class MoArray {
2 private:
3     struct Query {
4         int l, r, idx;
5         int bnd;
6
7         bool operator<(const Query& other) const {
8             if (bnd != other.bnd)
9                 return bnd < other.bnd;
10            return (bnd & 1) ? (r < other.r) : (r > other.r);
11        }
12    };
13
14    int n;
15    int block_size;
16    vector<int> a;
17
18    // --- PROBLEM SPECIFIC STATE ---
19    long long ans;
20    vector<int> freq;
21
22    inline void add(int idx) {
23        if (freq[a[idx]] == 0) ans++;
24        freq[a[idx]]++;
25    }
26
27    inline void remove(int idx) {
28        freq[a[idx]]--;
29        if (freq[a[idx]] == 0) ans--;
30    }
31
32    inline long long get_answer() const {
33        return ans;
34    }
35    // -----
36
37 public:
38     MoArray(const vector<int>& arr) : n(arr.size()), a(arr) {
39         int max_val = 0;
40         for (int x : a) {
41             max_val = max(max_val, x);
42         }
43         freq.assign(max_val + 1, 0);
44         ans = 0;
45     }
46
47     vector<long long> solve(const vector<pair<int, int>>& raw_queries)
48     {
49         int q = raw_queries.size();
50         if (q == 0) return {};
51
52         block_size = max(1, (int)(n / sqrt(q)));
53         vector<Query> queries(q);
54
55         for (int i = 0; i < q; ++i) {
56             queries[i].l = raw_queries[i].first;
57             queries[i].r = raw_queries[i].second;
58             queries[i].idx = i;
59             queries[i].bnd = queries[i].l / block_size;
60         }
61
62         sort(queries.begin(), queries.end());
63
64         vector<long long> answers(q);
65         int cur_l = 0, cur_r = -1;
66
67         for (const Query& qry : queries) {
68             while (cur_l > qry.l) add(--cur_l);
69             while (cur_r < qry.r) add(++cur_r);
70             while (cur_l < qry.l) remove(cur_l++);
71             while (cur_r > qry.r) remove(cur_r--);
72             answers[qry.idx] = get_answer();
73         }
74
75         return answers;
76 };

```

2.6.2 MoOnSubtrees

```

1 class MoSubtree {
2 private:
3     struct Query {
4         int l, r, idx;
5         int bnd;
6
7         bool operator<(const Query& other) const {
8             if (bnd != other.bnd) return bnd < other.bnd;
9             return (bnd & 1) ? (r < other.r) : (r > other.r);
10        }
11    };
12
13    int n;
14    int block_size;

```

```

15     vector<vector<int>> adj;
16     vector<int> in, out, flat, clr;
17
18     // --- PROBLEM SPECIFIC STATE ---
19     int mx_freq;
20     vector<int> freq;
21     vector<long long> sum_freq;
22
23     inline void add(int node_idx) {
24         int c = clr[node_idx];
25         sum_freq[freq[c]] -= c;
26         freq[c]++;
27         sum_freq[freq[c]] += c;
28         if (freq[c] > mx_freq) mx_freq = freq[c];
29     }
30
31     inline void remove(int node_idx) {
32         int c = clr[node_idx];
33         sum_freq[freq[c]] -= c;
34         freq[c]--;
35         sum_freq[freq[c]] += c;
36         while (mx_freq > 0 && sum_freq[mx_freq] == 0) mx_freq--;
37     }
38
39     inline long long get_answer() const {
40         return sum_freq[mx_freq];
41     }
42     // -----
43
44     void dfs(int u, int p) {
45         in[u] = flat.size();
46         flat.push_back(u);
47         for (int v : adj[u]) {
48             if (v == p) continue;
49             dfs(v, u);
50         }
51         out[u] = flat.size() - 1;
52     }
53
54 public:
55     MoSubtree(int n, int max_val, const vector<int>& colors)
56         : n(n), adj(n + 1), in(n + 1), out(n + 1), clr(colors) {
57         mx_freq = 0;
58         freq.assign(max_val + 1, 0);
59         sum_freq.assign(n + 1, 0);
60     }
61
62     void add_edge(int u, int v) {
63         adj[u].push_back(v);
64         adj[v].push_back(u);
65     }
66
67     vector<long long> solve(int root, const vector<int>& query_nodes) {
68         flat.reserve(n);
69         dfs(root, 0);
70
71         int q = query_nodes.size();
72         if (q == 0) return {};
73
74         block_size = max(1, (int)(n / sqrt(q)));
75         vector<Query> queries(q);
76
77         for (int i = 0; i < q; ++i) {
78             int u = query_nodes[i];
79             queries[i].l = in[u];
80             queries[i].r = out[u];
81             queries[i].idx = i;
82             queries[i].bnd = queries[i].l / block_size;
83         }
84
85         sort(queries.begin(), queries.end());
86
87         vector<long long> answers(q);
88         int cur_l = 0, cur_r = -1;
89
90         for (const Query& qry : queries) {
91             while (cur_l > qry.l) add(flat[--cur_l]);
92             while (cur_r < qry.r) add(flat[++cur_r]);
93             while (cur_l < qry.l) remove(flat[cur_l++]);
94             while (cur_r > qry.r) remove(flat[cur_r--]);
95             answers[qry.idx] = get_answer();
96         }
97
98         return answers;
99     }
100 };

```

2.6.3 MoOnPaths

```

1 class MoTreePath {
2 private:
3     int n, LOG, block_size;
4     vector<vector<int>> adj, anc;
5     vector<int> depth, in, out, flat;
6     vector<long long> up, a;
7     vector<bool> exist;
8
9     // --- PROBLEM SPECIFIC STATE ---
10    long long ans;
11    vector<deque<int>> dep; // Tracks nodes by depth
12
13    void add(int idx) {
14        if (!dep[depth[idx]].empty()) {
15            ans += 1ll * a[dep[depth[idx]].back()] * a[idx];
16        }
17        dep[depth[idx]].push_back(idx);
18    }
19
20    void remove(int idx) {
21        if (dep[depth[idx]].size() == 2) {
22            ans -= a[*dep[depth[idx]].begin()] * a[*next(dep[depth[idx]]
23                ].begin())];
24        }
25        if (dep[depth[idx]].front() == idx) {

```



```

25     dep[depth[idx]].pop_front();
26 } else {
27     dep[depth[idx]].pop_back();
28 }
29 }
30
31 void update(int idx) {
32     int node = flat[idx];
33     if (exist[node]) {
34         remove(node);
35     } else {
36         add(node);
37     }
38     exist[node] = !exist[node];
39 }
40
41 inline long long get_answer() const {
42     return ans;
43 }
44 // -----
45
46 void dfs(int u, int p = -1) {
47     if (p != -1) {
48         anc[u][0] = p;
49         up[u] = a[u] * a[u] + up[p];
50     } else {
51         depth[u] = 0;
52         up[u] = a[u] * a[u];
53     }
54     in[u] = flat.size();
55     flat.push_back(u);
56
57     for (int v : adj[u]) {
58         if (v == p) continue;
59         depth[v] = depth[u] + 1;
60         dfs(v, u);
61     }
62
63     out[u] = flat.size();
64     flat.push_back(u); // Push again for Eulerian tour
65 }
66
67 void build_lca() {
68     for (int k = 1; k < LOG; k++) {
69         for (int i = 1; i <= n; i++) {
70             anc[i][k] = anc[anc[i][k - 1]][k - 1];
71         }
72     }
73 }
74
75 int lift(int x, int k) {
76     for (int bit = 0; bit < LOG; bit++) {
77         if ((1 << bit) & k) x = anc[x][bit];
78     }
79     return x;
80 }
81
82 int lca(int u, int v) {
83     if (depth[u] < depth[v]) swap(u, v);
84     u = lift(u, depth[u] - depth[v]);
85     if (u == v) return u;
86     for (int bit = LOG - 1; bit >= 0; bit--) {
87         if (anc[u][bit] != anc[v][bit]) {
88             u = anc[u][bit];
89             v = anc[v][bit];
90         }
91     }
92     return anc[u][0];
93 }
94
95 public:
96 struct Query {
97     int l, r, idx, x = -1;
98     int bnd;
99
100     bool operator<(const Query& other) const {
101         if (bnd != other.bnd) return bnd < other.bnd;
102         return (bnd & 1) ? (r < other.r) : (r > other.r);
103     }
104 };
105
106 MoTreePath(int n_nodes, const vector<long long>& node_values)
107 : n(n_nodes), a(node_values), LOG(log2(n_nodes) + 2) {
108
109     adj.assign(n + 1, vector<int>());
110     anc.assign(n + 1, vector<int>(LOG, 0));
111     depth.assign(n + 1, 0);
112     in.assign(n + 1, 0);
113     out.assign(n + 1, 0);
114     up.assign(n + 1, 0);
115     exist.assign(n + 1, false);
116     dep.assign(n + 1, deque<int>());
117     ans = 0;
118 }
119
120 void add_edge(int u, int v) {
121     adj[u].push_back(v);
122     adj[v].push_back(u);
123 }
124
125 vector<long long> solve(int root, vector<Query>& queries) {
126     flat.clear();
127     dfs(root);
128     build_lca();
129
130     int q = queries.size();
131     if (q == 0) return {};
132
133     block_size = max(1, (int)(flat.size() / sqrt(q)));
134
135     for (auto& qry : queries) {
136         qry.bnd = qry.l / block_size;
137     }
138 }

```

```

139     sort(queries.begin(), queries.end());
140
141     vector<long long> answers(q);
142     int cur_l = 0, cur_r = -1;
143
144     for (const Query& qry : queries) {
145         while (cur_l < qry.l) update(cur_l++);
146         while (cur_r > qry.r) update(cur_r--);
147         while (cur_l > qry.l) update(--cur_l);
148         while (cur_r < qry.r) update(++cur_r);
149
150         answers[qry.idx] = get_answer() + (qry.x != -1 ? up[qry.x]
151             : 0);
152     }
153     return answers;
154 };

```

2.7 Balanced Binary Search Tree (BBST)

2.7.1 Treap

```

1 mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
2
3 template <typename T>
4 struct Treap {
5     struct Node {
6         T key, val, sum;
7         T lazy = 0; // 1. Added lazy tag
8         int sz = 1;
9         uint64_t p = rng();
10        Node *l = nullptr, *r = nullptr;
11
12        Node(T k, T v = T()) : key(k), val(v), sum(v) {}
13    } * root = nullptr;
14
15    ~Treap() { destroy(root); }
16
17    void destroy(Node* t) {
18        if (!t) return;
19        destroy(t->l);
20        destroy(t->r);
21        delete t;
22    }
23
24    int sz(Node* t) { return t ? t->sz : 0; }
25    T sub_val(Node* t) { return t ? t->sum : 0; }
26
27    // 2. The Push Function: Propagates pending updates to children
28    void push(Node* t) {
29        if (!t || t->lazy == 0) return;
30
31        if (t->l) {
32            t->l->key += t->lazy; // Shift the key (critical for the DP solution)
33            t->l->val += t->lazy; // Shift the value
34            t->l->sum += t->lazy * sz(t->l); // Update subtree sum
35            t->l->lazy += t->lazy; // Pass the tag down
36        }
37        if (t->r) {
38            t->r->key += t->lazy;
39            t->r->val += t->lazy;
40            t->r->sum += t->lazy * sz(t->r);
41            t->r->lazy += t->lazy;
42        }
43        t->lazy = 0; // Clear the tag on the current node
44    }
45
46    Node* update(Node* t) {
47        if (!t) {
48            t->sz = 1 + sz(t->l) + sz(t->r);
49            t->sum = t->val + sub_val(t->l) + sub_val(t->r);
50        }
51        return t;
52    }
53
54    Node* merge(Node* l, Node* r) {
55        push(l); // 3. Push before making routing decisions
56        push(r);
57        if (!l || !r) return l ? l : r;
58        if (l->p > r->p) {
59            l->r = merge(l->r, r);
60            return update(l);
61        }
62        r->l = merge(l, r->l);
63        return update(r);
64    }
65
66    pair<Node*, Node*> split(Node* t, T k) {
67        if (!t) return {nullptr, nullptr};
68        push(t); // 3. Push before evaluating keys for splits
69        if (t->key <= k) {
70            auto [l, r] = split(t->r, k);
71            t->r = l;
72            return {update(t), r};
73        }
74        auto [l, r] = split(t->l, k);
75        t->l = r;
76        return {l, update(t)};
77    }
78
79    // --- Core Operations ---
80
81    // Range Update Method
82    void add_range(T l_val, T r_val, T add_val) {
83        if (l_val > r_val) return;
84
85        auto [left_mid, right] = split(root, r_val);
86        auto [left, mid] = split(left_mid, l_val - 1);
87
88        // Apply the lazy tag to the entire isolated segment
89        if (mid) {
90            mid->key += add_val;
91            mid->val += add_val;

```

```

92         mid->sum += add_val * sz(mid);
93         mid->lazy += add_val;
94     }
95
96     root = merge(merge(left, mid), right);
97 }
98
99 bool search(T x) {
100     Node* curr = root;
101     while (curr) {
102         push(curr); // Push during traversal
103         if (curr->key == x) return true;
104         curr = (x < curr->key) ? curr->l : curr->r;
105     }
106     return false;
107 }
108
109 void insert(T x, T val = T()) {
110     auto [l, r] = split(root, x);
111     auto [l2, r2] = split(l, x - 1);
112     if (!r2) {
113         r2 = new Node(x, val);
114     } else {
115         push(r2); // Push before modifying
116         r2->val += val;
117         update(r2);
118     }
119     root = merge(merge(l2, r2), r);
120 }
121
122 void erase(T x) {
123     auto [l, r] = split(root, x);
124     auto [l2, r2] = split(l, x - 1);
125     destroy(r2);
126     root = merge(l2, r);
127 }
128
129 // --- Utility Operations ---
130
131 Node* find_by_order(Node* t, int k) {
132     if (!t) return nullptr;
133     push(t); // Push before accessing children sizes
134     int left_sz = sz(t->l);
135     if (k < left_sz) return find_by_order(t->l, k);
136     if (k == left_sz) return t;
137     return find_by_order(t->r, k - left_sz - 1);
138 }
139 Node* find_by_order(int k) { return find_by_order(root, k); }
140
141 int order_of_key(Node* t, T k) {
142     if (!t) return 0;
143     push(t); // Push before comparing keys
144     if (t->key >= k) return order_of_key(t->l, k);
145     return sz(t->l) + 1 + order_of_key(t->r, k);
146 }
147 int order_of_key(T k) { return order_of_key(root, k); }
148
149 void print(Node* t) {
150     if (!t) return;
151     push(t); // Push before printing to get true values
152     print(t->l);
153     cout << t->key << " ";
154     print(t->r);
155 }
156 void print() {
157     print(root);
158     cout << '\n';
159 }
160 };

```

2.7.2 ImplicitTreap

```

1 mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
2
3 template <typename T>
4 struct ImplicitTreap {
5     struct Node {
6         T val, sum;
7         T lazy = 0; // Lazy tag for range additions
8         bool rev = false; // Lazy tag for range reversals
9         int sz = 1;
10        uint64_t p = rng();
11        Node *l = nullptr, *r = nullptr;
12
13        Node(T v = T()) : val(v), sum(v) {}
14    } * root = nullptr;
15
16    // Default Constructor
17    ImplicitTreap() = default;
18
19    // Vector Constructor - Builds treap in O(N) time
20    ImplicitTreap(const vector<T>& a) {
21        build(a);
22    }
23
24    ~ImplicitTreap() { destroy(root); }
25
26    void destroy(Node* t) {
27        if (!t) return;
28        destroy(t->l);
29        destroy(t->r);
30        delete t;
31    }
32
33    int sz(Node* t) { return t ? t->sz : 0; }
34    T sub_val(Node* t) { return t ? t->sum : 0; }
35
36    // Propagates pending lazy updates to children
37    void push(Node* t) {
38        if (!t) return;
39
40        // 1. Process pending reversal
41        if (t->rev) {
42            swap(t->l, t->r);

```

```

43        if (t->l) t->l->rev ^= true;
44        if (t->r) t->r->rev ^= true;
45        t->rev = false;
46    }
47
48    // 2. Process pending value addition
49    if (t->lazy != 0) {
50        if (t->l) {
51            t->l->val += t->lazy;
52            t->l->sum += t->lazy * sz(t->l);
53            t->l->lazy += t->lazy;
54        }
55        if (t->r) {
56            t->r->val += t->lazy;
57            t->r->sum += t->lazy * sz(t->r);
58            t->r->lazy += t->lazy;
59        }
60        t->lazy = 0; // Clear the tag
61    }
62 }
63
64 Node* update(Node* t) {
65     if (t) {
66         t->sz = 1 + sz(t->l) + sz(t->r);
67         t->sum = t->val + sub_val(t->l) + sub_val(t->r);
68     }
69     return t;
70 }
71
72 // --- O(N) Linear Time Build ---
73 void build(const vector<T>& a) {
74     destroy(root);
75     root = nullptr;
76     if (a.empty()) return;
77
78     vector<Node*> st;
79     for (const T& val : a) {
80         Node* curr = new Node(val);
81         Node* last = nullptr;
82
83         // Maintain max-heap property on random priority 'p'
84         while (!st.empty() && st.back()->p < curr->p) {
85             last = st.back();
86             st.pop_back();
87         }
88
89         curr->l = last;
90         if (!st.empty()) {
91             st.back()->r = curr;
92         }
93         st.push_back(curr);
94     }
95
96     // Post-order pass to compute subtree sizes and sums in O(N)
97     auto recompute = [&](auto& self, Node* t) -> void {
98         if (!t) return;
99         self(self, t->l);
100        self(self, t->r);
101        update(t);
102    };
103
104     root = st.front(); // Bottom of stack is the root
105     recompute(recompute, root);
106 }
107
108 // --- Dynamic Array Operations ---
109
110 Node* merge(Node* l, Node* r) {
111     push(l);
112     push(r);
113     if (!l || !r) return l ? l : r;
114     if (l->p > r->p) {
115         l->r = merge(l->r, r);
116         return update(l);
117     }
118     r->l = merge(l, r->l);
119     return update(r);
120 }
121
122 pair<Node*, Node*> split(Node* t, int k) {
123     if (!t) return {nullptr, nullptr};
124     push(t);
125
126     int left_sz = sz(t->l);
127     if (left_sz < k) {
128         auto [l, r] = split(t->r, k - left_sz - 1);
129         t->r = l;
130         return {update(t), r};
131     } else {
132         auto [l, r] = split(t->l, k);
133         t->l = r;
134         return {l, update(t)};
135     }
136 }
137
138 int size() { return sz(root); }
139
140 void insert(int idx, T val) {
141     auto [l, r] = split(root, idx);
142     Node* node = new Node(val);
143     root = merge(merge(l, node), r);
144 }
145
146 void erase(int idx) {
147     if (idx < 0 || idx >= sz(root)) return;
148     auto [left_mid, right] = split(root, idx + 1);
149     auto [left, mid] = split(left_mid, idx);
150     destroy(mid);
151     root = merge(left, right);
152 }
153
154 void reverse_range(int l, int r) {
155     if (l >= r || l < 0 || r >= sz(root)) return;
156

```

```

157     auto [left_mid, right] = split(root, r + 1);
158     auto [left, mid] = split(left_mid, 1);
159
160     if (mid) mid->rev ^= true;
161
162     root = merge(merge(left, mid), right);
163 }
164
165 void add_range(int l, int r, T add_val) {
166     if (l > r || l < 0 || r >= sz(root)) return;
167
168     auto [left_mid, right] = split(root, r + 1);
169     auto [left, mid] = split(left_mid, 1);
170
171     if (mid) {
172         mid->val += add_val;
173         mid->sum += add_val * sz(mid);
174         mid->lazy += add_val;
175     }
176
177     root = merge(merge(left, mid), right);
178 }
179
180 T query_range(int l, int r) {
181     if (l > r || l < 0 || r >= sz(root)) return T();
182
183     auto [left_mid, right] = split(root, r + 1);
184     auto [left, mid] = split(left_mid, 1);
185
186     T ans = sub_val(mid);
187
188     root = merge(merge(left, mid), right);
189     return ans;
190 }
191
192 T get(int idx) {
193     return query_range(idx, idx);
194 }
195
196 void print(Node* t) {
197     if (!t) return;
198     push(t);
199     print(t->l);
200     cout << t->val << " ";
201     print(t->r);
202 }
203
204 void print() {
205     print(root);
206     cout << '\n';
207 }
208 };

```

2.8 Policy Based Data Structures

2.8.1 OrderedSet

```

1 #include <ext/pb_ds/assoc_container.hpp>
2 #include <ext/pb_ds/tree_policy.hpp>
3
4 using namespace __gnu_pbds;
5 template<class t> using ordered_set = tree<t, null_type, less<t>,
6     rb_tree_tag, tree_order_statistics_node_update>;

```

2.9 Monotonic Data Structures

2.9.1 TwoStackQueue

```

1 struct TwoStackQ {
2     struct Node {
3         ll mx = -llinf, mn = llinf, val;
4
5         Node(): val(0) {
6         }
7
8         Node(ll x): mx(x), mn(x), val(x) {
9         }
10    };
11
12    stack<Node> a, b;
13
14    int size() { return a.size() + b.size(); }
15
16    void mrg(Node& a, Node& b) {
17        a.mn = min(a.mn, b.mn);
18        a.mx = max(a.mx, b.mx);
19    }
20
21    void push(ll val) {
22        auto nd = Node(val);
23        if (!a.empty()) mrg(nd, a.top());
24        a.push(nd);
25    }
26
27    void move() {
28        while (!a.empty()) {
29            auto nd = Node(a.top().val);
30            if (!b.empty()) mrg(nd, b.top());
31            b.push(nd), a.pop();
32        }
33    }
34
35    Node get() {
36        Node res;
37        if (!b.empty()) mrg(res, b.top());
38        if (!a.empty()) mrg(res, a.top());
39        return res;
40    }
41
42    void pop() {
43        if (b.empty()) move();
44        if (!b.empty()) b.pop();
45    }
46 };

```

3 Graph Theory

3.1 Graph Traversal

3.1.1 TopoSort

```

1 struct TopoSort {
2     int n;
3     vector<vector<int>> adj;
4     vector<int> in_degree, order;
5
6     TopoSort(int n) : n(n), adj(n + 1), in_degree(n + 1, 0) {}
7
8     void add_edge(int u, int v) {
9         adj[u].push_back(v);
10        in_degree[v]++;
11    }
12
13    bool solve() {
14        queue<int> q;
15        for (int i = 1; i <= n; ++i) {
16            if (in_degree[i] == 0) q.push(i);
17        }
18
19        while (!q.empty()) {
20            int u = q.front();
21            q.pop();
22            order.push_back(u);
23
24            for (int v : adj[u]) {
25                if (--in_degree[v] == 0) {
26                    q.push(v);
27                }
28            }
29        }
30        return order.size() == n; // Returns false if there is a cycle
31    }
32 };

```

3.2 Shortest Paths

3.2.1 Dijkstra

```

1 template<typename T>
2 struct Dijkstra {
3     const T INF = numeric_limits<T>::max();
4     int n;
5     vector<vector<pair<int, T>> adj;
6     vector<T> dist;
7     vector<int> parent;
8
9     Dijkstra(int n) : n(n), adj(n + 1), dist(n + 1, INF), parent(n + 1,
10        -1) {}
11
12    void add_edge(int u, int v, T w, bool directed = false) {
13        adj[u].push_back({v, w});
14        if (!directed) adj[v].push_back({u, w});
15    }
16
17    void solve(int source) {
18        fill(dist.begin(), dist.end(), INF);
19        fill(parent.begin(), parent.end(), -1);
20        priority_queue<pair<T, int>, vector<pair<T, int>>, greater<pair<
21            T, int>>> pq;
22
23        dist[source] = 0;
24        pq.push({0, source});
25
26        while (!pq.empty()) {
27            auto [d, u] = pq.top();
28            pq.pop();
29
30            if (d > dist[u]) continue;
31
32            for (auto& [v, w] : adj[u]) {
33                if (dist[u] + w < dist[v]) {
34                    dist[v] = dist[u] + w;
35                    parent[v] = u;
36                    pq.push({dist[v], v});
37                }
38            }
39        }
40    };

```

3.2.2 FloydWarshall

```

1 template<typename T>
2 struct FloydWarshall {
3     const T INF = numeric_limits<T>::max() / 2; // Prevent overflow on
4         addition
5     int n;
6     vector<vector<T>> dist;
7
8     FloydWarshall(int n) : n(n), dist(n + 1, vector<T>(n + 1, INF)) {
9         for (int i = 0; i <= n; ++i) dist[i][i] = 0;
10    }
11
12    void add_edge(int u, int v, T w, bool directed = false) {
13        dist[u][v] = min(dist[u][v], w);
14        if (!directed) dist[v][u] = min(dist[v][u], w);
15    }
16
17    void solve() {
18        for (int k = 1; k <= n; ++k) {
19            for (int i = 1; i <= n; ++i) {
20                for (int j = 1; j <= n; ++j) {
21                    if (dist[i][k] < INF && dist[k][j] < INF) {
22                        dist[i][j] = min(dist[i][j], dist[i][k] + dist[

```

```

23     }
24 }
25 }
26 }
27 };

```

3.3 Graph Connectivity

3.3.1 BridgesAndCutEdges

```

1 int n;
2 vector<int> adj[N];
3
4 bool vis[N];
5 int in[N], low[N];
6 int timer;
7
8 set<pair<int, int>> bridges;
9
10 void dfs(int u, int p = 0) {
11     vis[u] = true;
12     in[u] = low[u] = timer++;
13     for (int v: adj[u]) {
14         if (v == p) continue;
15         if (vis[v]) {
16             low[u] = min(low[u], in[v]);
17         } else {
18             dfs(v, u);
19             low[u] = min(low[u], low[v]);
20             if (low[v] > in[u]) {
21                 /// (u, v) is a Bridge
22                 bridges.insert({u, v});
23                 bridges.insert({v, u});
24             }
25         }
26     }
27 }
28
29 void find_bridges() {
30     timer = 0;
31     fill(vis, vis + n + 1, false);
32     for (int i = 1; i <= n; ++i) {
33         if (!vis[i])
34             dfs(i);
35     }
36 }

```

3.3.2 ArticulationPoints

```

1 int n;
2 vector<int> adj[N];
3
4 bool vis[N];
5 int in[N], low[N];
6 int timer;
7
8 bool cutPoint[N];
9
10 void dfs(int u, int p = -1) {
11     vis[u] = true;
12     in[u] = low[u] = timer++;
13     int children = 0;
14     for (int v: adj[u]) {
15         if (v == p) continue;
16         if (vis[v]) {
17             low[u] = min(low[u], in[v]);
18         } else {
19             dfs(v, u);
20             low[u] = min(low[u], low[v]);
21             if (low[v] >= in[u] && p != -1) {
22                 /// u is a Cut Point
23                 cutPoint[u] = true;
24             }
25             children++;
26         }
27     }
28     if (p == -1 && children > 1) {
29         /// u is a Cut Point
30         cutPoint[u] = true;
31     }
32 }
33
34 bool connected = true;
35
36 void find_cut_points() {
37     timer = 0;
38     fill(vis, vis + n + 1, false);
39     for (int i = 1; i <= n; ++i) {
40         if (!vis[i]) {
41             if (i != 1)
42                 connected = false;
43             dfs(i);
44         }
45     }
46 }

```

3.3.3 SCC_Tarjan

```

1 /// assuming nodes are zero based
2 struct SCC {
3     vector<vector<int>> adj, adjRev, comps;
4     vector<pair<int, int>> edges;
5     vector<int> revOut, compOf;
6     vector<bool> vis;
7     int N;
8
9     void init(int n) {
10         N = n;
11         adj.resize(n);
12         adjRev.resize(n);
13         vis.resize(n);
14         compOf.resize(n);
15     }

```

```

16
17 void addEdge(int u, int v) {
18     edges.pb(make_pair(u, v));
19     adj[u].pb(v);
20     adjRev[v].pb(u);
21 }
22
23 void dfs1(int u) {
24     vis[u] = true;
25     for (auto v: adj[u])
26         if (!vis[v])
27             dfs1(v);
28     revOut.pb(u);
29 }
30
31 void dfs2(int u) {
32     vis[u] = true;
33     comps.back().pb(u);
34     compOf[u] = comps.size() - 1;
35     for (auto v: adjRev[u])
36         if (!vis[v]) dfs2(v);
37 }
38
39 void gen() {
40     fill(all(vis), false);
41     for (int i = 0; i < N; ++i) {
42         if (!vis[i])
43             dfs1(i);
44     }
45     reverse(all(revOut));
46     fill(all(vis), false);
47     for (auto node: revOut) {
48         if (vis[node]) continue;
49         comps.pb({});
50         dfs2(node);
51     }
52 }
53
54 vector<vector<int>> generateCondensedGraph() {
55     vector<vector<int>> adjCon(comps.size());
56     for (auto edge: edges)
57         if (compOf[edge.st] != compOf[edge.nd])
58             adjCon[compOf[edge.st]].pb(compOf[edge.nd]);
59     return adjCon;
60 }
61 };

```

3.4 Satisfiability

3.4.1 2SAT

```

1 struct TwoSat {
2     int N;
3     vector<pair<int, int>> edges;
4
5     TwoSat(int _N = 0) {
6         init(_N);
7     }
8
9     void init(int _N = 0) {
10         N = _N;
11     }
12
13     int addVar() { return N++; }
14
15     /// x or y, edges will be refined in the end
16     void either(int x, int y) {
17         x = max(2 * x, -1 - 2 * x);
18         y = max(2 * y, -1 - 2 * y);
19         edges.pb({x, y});
20     }
21
22     void implies(int x, int y) {
23         either(-x, y);
24     }
25
26     void must(int x) {
27         either(x, x);
28     }
29
30     void XOR(int x, int y) {
31         either(x, y);
32         either(-x, -y);
33     }
34
35     vector<bool> solve(int _N = -1) {
36         if (_N != -1) N = _N;
37         SCC scc;
38         scc.init(2 * N);
39         for (auto e: edges) {
40             scc.addEdge(e.st ^ 1, e.nd);
41             scc.addEdge(e.nd ^ 1, e.st);
42         }
43         scc.gen();
44         for (int i = 0; i < 2 * N; ++i) {
45             if (scc.compOf[i] == scc.compOf[i ^ 1]) return {};
46         }
47         vector<vector<int>> comps = scc.comps;
48         reverse(all(comps));
49         vector<int> compOf(2 * N);
50         for (int i = 0; i < comps.size(); ++i) {
51             for (auto e: comps[i])
52                 compOf[e] = i;
53         }
54         vector<int> tmp(comps.size());
55         for (int i = 0; i < comps.size(); ++i) {
56             if (!tmp[i]) {
57                 tmp[i] = 1;
58                 for (auto e: comps[i])
59                     tmp[compOf[e ^ 1]] = -1;
60             }
61         }
62         vector<bool> ans(N);
63         for (int i = 0; i < N; ++i)

```

```

64     ans[i] = tmp[comp0f[2 * i]] == 1;
65     return ans;
66 }
67
68 void atMostOne(const vector<int> &li) {
69     if (li.size() <= 1) return;
70     int last = -li[i];
71     for (int i = 2; i < li.size(); i++) {
72         int next = addVar();
73         implies(li[i], last);
74         either(last, next);
75         implies(li[i], next);
76         last = -next;
77     }
78     implies(li[0], last);
79 }
80 };

```

3.5 Eulerian Path

3.5.1 EulerianPath

```

1 struct EulerianPath {
2     int n;
3     vector<vector<pair<int, int>>> adj; // {neighbor, edge_index}
4     vector<bool> used_edge;
5     vector<int> in_deg, out_deg;
6     vector<int> path;
7
8     EulerianPath(int n, int m) : n(n), adj(n + 1), used_edge(m, false),
9         in_deg(n + 1, 0), out_deg(n + 1, 0) {}
10
11     void addEdge(int u, int v, int edge_idx, bool directed = true) {
12         adj[u].push_back({v, edge_idx});
13         out_deg[u]++; in_deg[v]++;
14         if (!directed) {
15             adj[v].push_back({u, edge_idx});
16             out_deg[v]++; in_deg[u]++;
17         }
18     }
19
20     bool solve(int start_node) {
21         vector<int> ptr(n + 1, 0);
22         stack<int> st;
23         st.push(start_node);
24
25         while (!st.empty()) {
26             int u = st.top();
27             if (ptr[u] < adj[u].size()) {
28                 auto [v, id] = adj[u][ptr[u]++];
29                 if (!used_edge[id]) {
30                     used_edge[id] = true;
31                     st.push(v);
32                 }
33             } else {
34                 path.push_back(u);
35                 st.pop();
36             }
37         }
38         reverse(path.begin(), path.end());
39
40         // Check if all edges were visited
41         for (bool used : used_edge) {
42             if (!used) return false;
43         }
44         return true;
45     };

```

3.6 Flow and Min Cut

3.6.1 DinicMaxFlow

```

1 struct Dinic {
2     struct Edge {
3         int to, rev;
4         ll c, oc;
5         ll flow() { return max(oc - c, 0LL); } // if you need flows
6     };
7
8     vector<int> lvl, ptr, q;
9     vector<vector<Edge>> > adj;
10
11     Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
12
13     void addEdge(int a, int b, ll c, ll rcap = 0) {
14         adj[a].push_back({b, (int) adj[b].size(), c, c});
15         adj[b].push_back({a, (int) adj[a].size() - 1, rcap, rcap});
16     }
17
18     ll dfs(int v, int t, ll f) {
19         if (v == t || !f) return f;
20         for (int &i = ptr[v]; i < (int) adj[v].size(); i++) {
21             Edge &e = adj[v][i];
22             if (lvl[e.to] == lvl[v] + 1)
23                 if (ll p = dfs(e.to, t, min(f, e.c))) {
24                     e.c -= p, adj[e.to][e.rev].c += p;
25                     return p;
26                 }
27         }
28         return 0;
29     }
30
31     ll calc(int s, int t) {
32         ll flow = 0;
33         q[0] = s;
34         for (int L = 0; L < 31; L++)
35             do {
36                 // 'int L=30' maybe faster for random data
37                 lvl = ptr = vector<int>((int) q.size());
38                 int qi = 0, qe = lvl[s] = 1;
39                 while (qi < qe && !lvl[t]) {
40                     int v = q[qi++];

```

```

42         for (Edge e : adj[v])
43             if (!lvl[e.to] && e.c >> (30 - L))
44                 q[qi++] = e.to, lvl[e.to] = lvl[v] + 1;
45         }
46         while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
47     } while (lvl[t]);
48     return flow;
49 }
50
51 bool leftOfMinCut(int a) { return lvl[a] != 0; }
52 };

```

3.6.2 MinCostMaxFlow

```

1 struct Edge {
2     int to;
3     int cost;
4     int cap, flow, backEdge;
5 };
6
7 struct MCMF
8 {
9
10     const int inf = 1000000010;
11     int n;
12     vector<vector<Edge>> g;
13
14     MCMF(int _n) {
15         n = _n + 1;
16         g.resize(n);
17     }
18
19     void addEdge(int u, int v, int cap, int cost) {
20         Edge e1 = {v, cost, cap, 0, (int) g[v].size()};
21         Edge e2 = {u, -cost, 0, 0, (int) g[u].size()};
22         g[u].push_back(e1);
23         g[v].push_back(e2);
24     }
25
26     pair<int, int> minCostMaxFlow(int s, int t) {
27         int flow = 0;
28         int cost = 0;
29         vector<int> state(n), from(n), from_edge(n);
30         vector<int> d(n);
31         deque<int> q;
32         while (true) {
33             for (int i = 0; i < n; i++)
34                 state[i] = 2, d[i] = inf, from[i] = -1;
35             state[s] = 1;
36             q.clear();
37             q.push_back(s);
38             d[s] = 0;
39             while (!q.empty()) {
40                 int v = q.front();
41                 q.pop_front();
42                 state[v] = 0;
43                 for (int i = 0; i < (int) g[v].size(); i++) {
44                     Edge e = g[v][i];
45                     if (e.flow >= e.cap || (d[e.to] <= d[v] + e.cost))
46                         continue;
47                     int to = e.to;
48                     d[to] = d[v] + e.cost;
49                     from[to] = v;
50                     from_edge[to] = i;
51                     if (state[to] == 1) continue;
52                     if (!state[to] || (!q.empty() && d[q.front()] > d[to]))
53                         q.push_front(to);
54                     else q.push_back(to);
55                     state[to] = 1;
56                 }
57             }
58             if (d[t] == inf) break;
59             int it = t, addflow = inf;
60             while (it != s) {
61                 addflow = min(addflow,
62                     g[from[it]][from_edge[it]].cap
63                     - g[from[it]][from_edge[it]].flow);
64                 it = from[it];
65             }
66             it = t;
67             while (it != s) {
68                 g[from[it]][from_edge[it]].flow += addflow;
69                 g[it][g[from[it]][from_edge[it]].backEdge].flow -=
70                     addflow;
71                 cost += g[from[it]][from_edge[it]].cost * addflow;
72                 it = from[it];
73             }
74             flow += addflow;
75             return {cost, flow};
76         }
77     };

```

3.7 Matching

3.7.1 HopcroftKarpMatching

```

1 struct HopcroftKarp {
2     vector<int> leftMatch, rightMatch, dist, cur;
3     vector<vector<int>> > a;
4     int n, m;
5
6     HopcroftKarp() {}
7
8     HopcroftKarp(int n, int m) {
9         this->n = n;
10        this->m = m;
11        a = vector<vector<int>> >(n);
12        leftMatch = vector<int>(m, -1);
13        rightMatch = vector<int>(n, -1);
14        dist = vector<int>(n, -1);
15        cur = vector<int>(n, -1);
16    }

```

```

17
18 void addEdge(int x, int y) {
19     a[x].push_back(y);
20 }
21
22 int bfs() {
23     int found = 0;
24     queue<int> q;
25     for (int i = 0; i < n; i++)
26         if (rightMatch[i] < 0) dist[i] = 0, q.push(i);
27         else dist[i] = -1;
28
29     while (!q.empty()) {
30         int x = q.front();
31         q.pop();
32         for (int i = 0; i < int(a[x].size()); i++) {
33             int y = a[x][i];
34             if (leftMatch[y] < 0) found = 1;
35             else if (dist[leftMatch[y]] < 0)
36                 dist[leftMatch[y]] = dist[x] + 1, q.push(leftMatch[
37
38         }
39     }
40
41     return found;
42 }
43
44 int dfs(int x) {
45     for (; cur[x] < int(a[x].size()); cur[x]++) {
46         int y = a[x][cur[x]];
47         if (leftMatch[y] < 0 || (dist[leftMatch[y]] == dist[x] + 1
48             && dfs(leftMatch[y]))) {
49             leftMatch[y] = x;
50             rightMatch[x] = y;
51             return 1;
52         }
53     }
54     return 0;
55 }
56
57 int maxMatching() {
58     int match = 0;
59     while (bfs()) {
60         for (int i = 0; i < n; i++) cur[i] = 0;
61         for (int i = 0; i < n; i++)
62             if (rightMatch[i] < 0) match += dfs(i);
63     }
64     return match;
65 }

```

4 Number Theory

4.1 Euclidean Algorithm

4.1.1 GCD_LCM

```

1 ll gcd(ll a, ll b) {
2     if (b == 0)
3         return a;
4     return gcd(b, a % b);
5     // return __gcd(a, b);
6 }
7
8 ll lcm(ll a, ll b) {
9     return (a / gcd(a, b)) * b;
10 }

```

4.1.2 ExtendedEuclid

```

1 long long extGCD(long long a, long long b, long long &x, long long &y)
2 {
3     if (b == 0) {
4         x = 1; y = 0;
5         return a;
6     }
7     long long x1, y1;
8     long long d = extGCD(b, a % b, x1, y1);
9     x = y1;
10    y = x1 - y1 * (a / b);
11    return d;
12 }
13
14 long long modInverse(long long a, long long m) {
15     long long x, y;
16     long long g = extGCD(a, m, x, y);
17     if (g != 1) return -1; // Inverse doesn't exist
18     return (x % m + m) % m;
19 }

```

4.2 Primes and Divisors

4.2.1 SieveOfEratosthenes

```

1 bool isPrime[N];
2
3 void seive(int n = N) {
4     //complexity: n*log(log(n))
5     memset(isPrime, true, sizeof isPrime);
6     isPrime[0] = false;
7     isPrime[1] = false;
8     for (int i = 2; i * i < n; i++) {
9         if (isPrime[i]) {
10             for (int j = i * i; j < n; j += i) {
11                 isPrime[j] = false;
12             }
13         }
14     }
15 }

```

4.2.2 SmallestPrimeFactor

```

1 vector<int> spf;

```

```

2 void seive(int n = N) {
3     //complexity: n*log(log(n))
4     spf.resize(n);
5     iota(all(spf), 0);
6     for (int i = 2; i * i < n; i++) {
7         if (spf[i] == i) {
8             for (int j = i * i; j < n; j += i) {
9                 spf[j] = min(spf[j], i);
10            }
11        }
12    }
13 }
14
15 vector<int> getPrimeFactors(int n) {
16     //complexity: log(n)
17     vector<int> ans;
18     while (n != 1) {
19         ans.pb(spf[n]);
20         n /= spf[n];
21     }
22     return ans;
23 }

```

4.2.3 DivisorsGeneration

```

1 template <typename T>
2 vector<T> getDivisors(T n) {
3     //complexity: sqrt(n)
4     vector<T> vec;
5     for (ll i = 1; i * i <= n; i++) {
6         if (n % i == 0) {
7             vec.push_back(i);
8             if (n / i != i)
9                 vec.push_back(n / i);
10        }
11    }
12    return vec;
13 }

```

4.2.4 PrimeFactorization

```

1 template <typename T>
2 vector<T> getPrimeFactors(T n) {
3     //complexity: sqrt(n)
4     vector<T> vec;
5     for (ll i = 2; i * i <= n; i++) {
6         while (n % i == 0) {
7             vec.push_back(i);
8             n /= i;
9         }
10    }
11    if (n != 1) vec.push_back(n);
12    return vec;
13 }

```

4.3 Modular Arithmetic

4.3.1 CRT

```

1 long long extGCD(long long a, long long b, long long &x, long long &y)
2 {
3     if (b == 0) {
4         x = 1; y = 0;
5         return a;
6     }
7     long long x1, y1;
8     long long d = extGCD(b, a % b, x1, y1);
9     x = y1;
10    y = x1 - y1 * (a / b);
11    return d;
12 }
13 // Solves x = a_i (mod m_i). Returns {x, lcm(m_1, ..., m_k)}.
14 pair<long long, long long> CRT(const vector<long long>& a, const vector
15     <long long>& m) {
16     long long res = a[0], lcm = m[0];
17     for (int i = 1; i < a.size(); i++) {
18         long long x, y;
19         long long g = extGCD(lcm, m[i], x, y);
20         if ((a[i] - res) % g != 0) return {-1, -1}; // No solution
21
22         long long step = m[i] / g;
23         long long k = ((a[i] - res) / g) % step * (x % step) % step;
24         res += k * lcm;
25         lcm *= step;
26         res = (res % lcm + lcm) % lcm;
27     }
28     return {res, lcm};
29 }

```

5 Combinatorics

6 Math

6.1 Modular Arithmetic

6.1.1 ModularArithmetic

```

1 int fact[N], inv[N], invFact[N]; ///used in nCr(), nPr(), calcFact()
2
3 int modSum(ll a, ll b) {
4     if (a < 0)
5         a += MOD;
6     if (b < 0)
7         b += MOD;
8     a += b;
9     if (a >= MOD)
10        a -= MOD;
11    return a;
12
13    a = (a % MOD + MOD) % MOD;
14    b = (b % MOD + MOD) % MOD;

```



```

15     return (a + b) % MOD;
16 }
17
18 int modProd(ll a, ll b) {
19     if (a < 0)
20         a += MOD;
21     if (b < 0)
22         b += MOD;
23     a *= b;
24     if (a >= MOD)
25         a %= MOD;
26     return a;
27
28     a = (a % MOD + MOD) % MOD;
29     b = (b % MOD + MOD) % MOD;
30     return (a * b) % MOD;
31 }
32
33
34 int modPow(int a, ll b) {
35     int result = 1;
36     while (b) {
37         if (b & 1)
38             result = modProd(result, a);
39         a = modProd(a, a);
40         b >>= 1;
41     }
42     return result;
43 }
44
45
46 int modInverse(int a) {
47     return modPow(a, MOD - 2);
48 }
49
50
51 int modDiv(int a, int b) {
52     return modProd(a, modInverse(b));
53 }
54
55 int nCr(int n, int r) {
56     if (r < 0 || r > n)
57         return 0;
58     // return modDiv(fact[n], modProd(fact[r], fact[n - r]));
59     return modProd(fact[n], modProd(invFact[r], invFact[n - r]));
60 }
61
62 int nPr(int n, int r) {
63     if (r < 0 || r > n)
64         return 0;
65     // return modDiv(fact[n], fact[n - r]);
66     return modProd(fact[n], invFact[n - r]);
67 }
68
69 void build() {
70     fact[0] = invFact[0] = 1;
71     for (int i = 1; i < N; i++) {
72         fact[i] = modProd(fact[i - 1], i);
73         invFact[i] = modDiv(invFact[i - 1], i);
74         inv[i] = modInverse(i);
75     }
76 }

```

6.1.2 ModularIntClass

```

1 struct Mint {
2     int v;
3
4     Mint(long long val = 0) {
5         v = int(val % MOD);
6         if (v < 0) v += MOD;
7     }
8
9     Mint operator+(const Mint& o) const { return Mint(v + o.v); }
10    Mint operator-(const Mint& o) const { return Mint(v - o.v); }
11    Mint operator*(const Mint& o) const { return Mint(1LL * v * o.v); }
12    Mint operator/(const Mint& o) const { return *this * o.inv(); }
13
14    Mint& operator+=(const Mint& o) {
15        v += o.v;
16        if (v >= MOD) v -= MOD;
17        return *this;
18    }
19
20    Mint& operator-=(const Mint& o) {
21        v -= o.v;
22        if (v < 0) v += MOD;
23        return *this;
24    }
25
26    Mint& operator*=(const Mint& o) {
27        v = int(1LL * v * o.v % MOD);
28        return *this;
29    }
30
31    Mint& operator/=(const Mint& o) {
32        v = int(1LL * v * o.inv().v % MOD);
33        return *this;
34    }
35
36    Mint pow(long long p) const {
37        Mint a = *this, res = 1;
38        while (p > 0) {
39            if (p & 1) res *= a;
40            a *= a;
41            p >>= 1;
42        }
43        return res;
44    }
45
46    Mint inv() const { return pow(MOD - 2); }
47
48    friend ostream& operator<<(ostream& os, const Mint& m) {
49        os << m.v;

```

```

50         return os;
51     }
52 };
53
54 Mint fact[N], inv[N], invFact[N];
55
56 Mint nCr(int n, int r) {
57     if (r < 0 || r > n)
58         return 0;
59     // return fact[n] / (fact[r] * fact[n - r]);
60     return fact[n] * invFact[r] * invFact[n - r];
61 }
62
63 Mint nPr(int n, int r) {
64     if (r < 0 || r > n)
65         return 0;
66     // return fact[n] / fact[n - r];
67     return fact[n] * invFact[n - r];
68 }
69
70 void build() {
71     fact[0] = invFact[0] = 1;
72     for (int i = 1; i < N; i++) {
73         fact[i] = fact[i - 1] * i;
74         invFact[i] = invFact[i - 1] / i;
75         inv[i] = Mint(i).inv();
76     }
77 }

```

6.2 Multiplicative Functions

6.2.1 EulerTotientPhi

```

1 int phi[N];
2
3 void calculatePhi() {
4     for (int i = 0; i < N; ++i) phi[i] = i & 1 ? i : i / 2;
5     for (int i = 3; i < N; i += 2)
6         if (phi[i] == i)
7             for (int j = i; j < N; j += i) phi[j] -= phi[j] / i;
8 }

```

6.3 Matrices

6.3.1 MatrixExponentiation

```

1 int modSum(ll a, ll b) {
2     if (a < 0)
3         a += MOD;
4     if (b < 0)
5         b += MOD;
6     a += b;
7     if (a >= MOD)
8         a -= MOD;
9     return a;
10 }
11
12 int modProd(ll a, ll b) {
13     if (a < 0)
14         a += MOD;
15     if (b < 0)
16         b += MOD;
17     a *= b;
18     if (a >= MOD)
19         a %= MOD;
20     return a;
21 }
22
23 typedef vector<vector<int>> matrix;
24
25 matrix operator*(const matrix& lhs, const matrix& rhs) {
26     int n = lhs.size();
27     int m = rhs[0].size();
28     int s1 = lhs[0].size(), s2 = rhs.size();
29     assert(s1 == s2);
30     matrix ret(n, vector<int>(m));
31     for (int i = 0; i < n; ++i)
32         for (int j = 0; j < s1; ++j)
33             for (int k = 0; k < m; ++k)
34                 ret[i][k] = modSum(ret[i][k], modProd(lhs[i][j], rhs[j][k]));
35     return ret;
36 }
37
38 matrix Identity(int n) {
39     matrix ret(n, vector<int>(n));
40     for (int i = 0; i < n; ++i) {
41         ret[i][i] = 1;
42     }
43     return ret;
44 }
45
46 matrix mat_power(matrix x, ll p) {
47     matrix res = Identity(x.size());
48     while (p) {
49         if (p & 1) res = (res * x);
50         x = (x * x);
51         p >>= 1;
52     }
53     return res;
54 }

```

6.3.2 FastMatrixPower

```

1 const int SZ = 2, mod = 1e9 + 7;
2
3 int modSum(ll a, ll b) {
4     if (a < 0)
5         a += mod;
6     if (b < 0)
7         b += mod;
8     a += b;
9     if (a >= mod)

```



```

10     a -= mod;
11     return a;
12 }
13
14 int modProd(ll a, ll b) {
15     if (a < 0)
16         a += mod;
17     if (b < 0)
18         b += mod;
19     a *= b;
20     if (a >= mod)
21         a %= mod;
22     return a;
23 }
24
25 typedef array<array<int, SZ>, SZ> matrix;
26
27 matrix operator*(const matrix &lhs, const matrix &rhs) {
28     matrix ret{};
29     for (int i = 0; i < SZ; ++i)
30         for (int j = 0; j < SZ; ++j)
31             for (int k = 0; k < SZ; ++k)
32                 ret[i][k] = modSum(ret[i][k], modProd(lhs[i][j], rhs[j][k]));
33     return ret;
34 }
35
36 matrix Identity(int n) {
37     matrix ret = {};
38     for (int i = 0; i < n; ++i) {
39         ret[i][i] = 1;
40     }
41     return ret;
42 }
43
44 matrix mat_power(matrix x, ll p) {
45     matrix res = Identity(x.size());
46     while (p) {
47         if (p & 1) res = (res * x);
48         x = (x * x);
49         p >>= 1;
50     }
51     return res;
52 }

```

6.4 Linear Algebra

6.4.1 LinearXorBasis

```

1
2
3 struct Basis {
4     vector<ll> basis;
5     int LOG, sz;
6
7     Basis(int log = 60) {
8         LOG = log;
9         sz = 0;
10        basis.resize(LOG);
11    }
12
13    bool insert(ll x) {
14        for (int i = LOG - 1; i >= 0; --i) {
15            if (x >> i & 1) continue;
16            if (basis[i]) {
17                x ^= basis[i];
18            } else {
19                basis[i] = x;
20                sz++;
21                return true;
22            }
23        }
24        return false;
25    }
26
27    ll getMax(ll x = 0) {
28        ll ret = x;
29        for (int i = LOG - 1; i >= 0; --i) {
30            ret = max(ret, ret ^ basis[i]);
31        }
32        return ret;
33    }
34 };

```

6.5 Convolution

6.5.1 FastFourierTransform

```

1 #define rep(aa, bb, cc) for(int aa = bb; aa < cc; aa++)
2 #define sz(a) (int)a.size()
3 typedef complex<double> C;
4 typedef vector<double> vd;
5 void fft(vector<C>& a) {
6     int n = sz(a), L = 31 - __builtin_clz(n);
7     static vector<complex<long double>> R(2, 1);
8     static vector<C> rt(2, 1); // (~ 10% faster if double)
9     for (static int k = 2; k < n; k *= 2) {
10         R.resize(n); rt.resize(n);
11         auto x = polar(1.0L, acos(-1.0L) / k);
12         rep(i, k, 2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
13     }
14     vi rev(n);
15     rep(i, 0, n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
16     rep(i, 0, n) if (i < rev[i]) swap(a[i], a[rev[i]]);
17     for (int k = 1; k < n; k *= 2)
18         for (int i = 0; i < n; i += 2 * k) rep(j, 0, k) {
19             // C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-rolled)
20             auto x = (double *)&rt[j+k], y = (double *)&a[i+j+k];
21             C z(x[0]*y[0] - x[1]*y[1], x[0]*y[1] + x[1]*y[0]);
22             a[i + j + k] = a[i + j] - z;
23             a[i + j] += z;

```

```

24         }
25     }
26
27 template<int M> vi convMod(const vi &a, const vi &b) {
28     if (a.empty() || b.empty()) return {};
29     vi res(sz(a) + sz(b) - 1);
30     int B=32-__builtin_clz(sz(res)), n=1<B, cut=int(sqrt(M));
31     vector<C> L(n), R(n), outs(n), outl(n);
32     rep(i, 0, sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut);
33     rep(i, 0, sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut);
34     fft(L), fft(R);
35     rep(i, 0, n) {
36         int j = -i & (n - 1);
37         outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
38         outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
39     }
40     fft(outl), fft(outs);
41     rep(i, 0, sz(res)) {
42         ll av = 1l(real(outl[i])+.5), cv = 1l(imag(outs[i])+.5);
43         ll bv = 1l(imag(outl[i])+.5) + 1l(real(outs[i])+.5);
44         res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
45     }
46     return res;
47 }

```

6.5.2 NumberTheoreticTransform

```

1 const int MOD = 998244353;
2 const int ROOT = 3; // Primitive root for 998244353
3
4 long long power(long long base, long long exp) {
5     long long res = 1;
6     base %= MOD;
7     while (exp > 0) {
8         if (exp % 2 == 1) res = (res * base) % MOD;
9         base = (base * base) % MOD;
10        exp /= 2;
11    }
12    return res;
13 }
14
15 long long modInverseFast(long long n) {
16     return power(n, MOD - 2);
17 }
18
19 void ntt(vector<long long>& a, bool invert) {
20     int n = a.size();
21     for (int i = 1, j = 0; i < n; i++) {
22         int bit = n >> 1;
23         for (; j & bit; bit >>= 1) j ^= bit;
24         j ^= bit;
25         if (i < j) swap(a[i], a[j]);
26     }
27     for (int len = 2; len <= n; len <= 1) {
28         long long wlen = power(ROOT, (MOD - 1) / len);
29         if (invert) wlen = modInverseFast(wlen);
30         for (int i = 0; i < n; i += len) {
31             long long w = 1;
32             for (int j = 0; j < len / 2; j++) {
33                 long long u = a[i + j], v = (a[i + j + len / 2] * w) % MOD;
34                 a[i + j] = u + v < MOD ? u + v : u + v - MOD;
35                 a[i + j + len / 2] = u - v >= 0 ? u - v : u - v + MOD;
36                 w = (w * wlen) % MOD;
37             }
38         }
39     }
40     if (invert) {
41         long long n_inv = modInverseFast(n);
42         for (long long& x : a) x = (x * n_inv) % MOD;
43     }
44 }

```

7 Strings

7.1 String Matching

7.1.1 KMP_PrefixFunction

```

1 vector<int> compute_pi(const string &s) {
2     int n = s.size();
3     vector<int> pi(n);
4     for (int i = 1; i < n; i++) {
5         int j = pi[i - 1];
6         while (j > 0 && s[i] != s[j])
7             j = pi[j - 1];
8         if (s[i] == s[j])
9             j++;
10        pi[i] = j;
11    }
12    return pi;
13 }
14
15 vector<int> find_matches(const string &text, const string &pat) {
16     int n = pat.length(), m = text.length();
17     string s = pat + "#" + text;
18     vector<int> pi = compute_pi(s), ans;
19     for (int i = n + 1; i <= n + m; i++) { /* n + 1 is where the text starts */
20         if (pi[i] == n) {
21             ans.push_back(i - 2 * n); /* i - (n - 1) - (n + 1) */
22         }
23     }
24     return ans;
25 }
26
27 void compute_automaton(string s, vector<vector<int>> &aut) {
28     s += '#';
29     int n = s.size();
30     vector<int> pi = compute_pi(s);
31     aut.assign(n, vector<int>(26));
32     for (int i = 0; i < n; i++) {
33         for (int c = 0; c < 26; c++) {

```

```

34         if (i > 0 && 'a' + c != s[i])
35             aut[i][c] = aut[pi[i - 1]][c];
36         else
37             aut[i][c] = i + ('a' + c == s[i]);
38     }
39 }
40 }

```

7.1.2 RabinKarp

```

1 vector<int> rabin_karp(string const &s, string const &t) {
2     const int p = 31;
3     const int m = 1e9 + 9;
4     int S = s.size(), T = t.size();
5
6     vector<ll> p_pow(max(S, T));
7     p_pow[0] = 1;
8     for (int i = 1; i < (int) p_pow.size(); i++)
9         p_pow[i] = (p_pow[i - 1] * p) % m;
10
11     vector<ll> h(T + 1, 0);
12     for (int i = 0; i < T; i++)
13         h[i + 1] = (h[i] + (t[i] - 'a' + 1) * p_pow[i]) % m;
14     ll h_s = 0;
15     for (int i = 0; i < S; i++)
16         h_s = (h_s + (s[i] - 'a' + 1) * p_pow[i]) % m;
17
18     vector<int> occurrences;
19     for (int i = 0; i + S - 1 < T; i++) {
20         ll cur_h = (h[i + S] + m - h[i]) % m;
21         if (cur_h == h_s * p_pow[i] % m)
22             occurrences.push_back(i);
23     }
24     return occurrences;
25 }

```

7.1.3 ZFunction

```

1 // z[i] is the length of the longest substring starting from s[i] which
2 // is also a prefix of s
3 vector<int> z_function(string s) {
4     int n = s.length();
5     vector<int> z(n);
6     int l = 0, r = 0;
7     for (int i = 1; i < n; i++) {
8         if (i < r) {
9             z[i] = min(r - i, z[i - l]);
10        }
11        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
12            z[i]++;
13        }
14        if (i + z[i] > r) {
15            l = i;
16            r = i + z[i];
17        }
18    }
19    return z;
20 }

```

7.2 Hashing

7.2.1 DoubleStringHashing

```

1 int modSum(ll a, ll b) {
2     ///complexity: 1
3     if (a < 0)
4         a += MOD;
5     if (b < 0)
6         b += MOD;
7     a += b;
8     if (a >= MOD)
9         a %= MOD;
10    return a;
11 }
12
13 int modProd(ll a, ll b) {
14     ///complexity: 1
15     if (a < 0)
16         a += MOD;
17     if (b < 0)
18         b += MOD;
19     a *= b;
20     if (a >= MOD)
21         a %= MOD;
22    return a;
23 }
24
25 int fastPow(int a, ll b) {
26     ///complexity: log(b)
27     if (b == 0) return 1;
28     int temp = fastPow(a, b / 2);
29     if (b % 2 == 0)
30         return modProd(temp, temp);
31     return modProd(modProd(a, temp), temp);
32 }
33
34
35 int modInverse(int a) {
36     ///complexity: log(mod)
37     return fastPow(a, MOD - 2);
38 }
39
40
41 struct Hash {
42     vector<pair<int, int>> hash_value = {{0, 0}};
43     const pair<int, int> p = {67, 97};
44     vector<pair<int, int>> p_pow = {{1, 1}};
45     vector<pair<int, int>> inv = {{1, 1}};
46
47     void compute(string const &s) {
48         for (char c: s) {
49             if (c >= 'a' && c <= 'z') {
50                 c = c - 'a' + 1;

```

```

51         } else if (c >= 'A' && c <= 'Z') {
52             c = c - 'A' + 30;
53         } else {
54             c = c - '0' + 60;
55         }
56         int x = modSum(hash_value.back().st, modProd(c, p_pow.back().st));
57         int a = modProd(p_pow.back().st, p.st);
58
59         int y = modSum(hash_value.back().nd, modProd(c, p_pow.back().nd));
60         int b = modProd(p_pow.back().nd, p.nd);
61
62         hash_value.emplace_back(x, y);
63         p_pow.emplace_back(a, b);
64         if (inv.size() == 1) {
65             inv.emplace_back(modInverse(a), modInverse(b));
66         } else {
67             inv.emplace_back(modProd(inv[1].st, inv.back().st),
68                               modProd(inv[1].nd, inv.back().nd));
69         }
70     }
71
72     pair<int, int> get(int l, int r) {
73         l++, r++;
74         pair<int, int> ans;
75         ans.st = modSum(hash_value[r].st, -hash_value[l - 1].st);
76         ans.st = modProd(ans.st, inv[l].st);
77
78         ans.nd = modSum(hash_value[r].nd, -hash_value[l - 1].nd);
79         ans.nd = modProd(ans.nd, inv[l].nd);
80
81         return ans;
82     }
83 }
84 };

```

7.3 Aho Corasick

7.3.1 AhoCorasickAutomaton

```

1 struct Mapper {
2     int operator()(char c) const { return c - 'a'; }
3 };
4
5 template <int ALPHABET, typename Mapper>
6 struct Aho {
7     struct Node {
8         array<int, ALPHABET> nxt;
9         int link = 0;
10        int terminal_idx = -1; // -1 if not a terminal, else idx of string insertion
11        int exit_link = 0; // Points to the nearest terminal suffix
12        int depth = 0; // Size of the prefix the node represents
13        Node() { nxt.fill(0); }
14    };
15
16    vector<Node> t;
17    vector<int> bfs_order;
18    Mapper get_idx; // Instance of the mapping function
19    int word_count = 0; // Tracks the order of inserted strings
20
21    // Constructor optionally accepts a custom mapper instance
22    Aho(Mapper mapper = Mapper()) : get_idx(mapper) {
23        t.emplace_back();
24    }
25
26    int insert(const string& p) {
27        int u = 0;
28        for (char c : p) {
29            int v = get_idx(c);
30            if (!t[u].nxt[v]) {
31                t[u].nxt[v] = t.size();
32                t.emplace_back();
33                t.back().depth = t[u].depth + 1; // New node's depth is parent's depth + 1
34            }
35            u = t[u].nxt[v];
36        }
37
38        // If this exact string hasn't been inserted before, assign it the current index
39        if (t[u].terminal_idx == -1) {
40            t[u].terminal_idx = word_count;
41        }
42        word_count++; // Increment for the next insertion
43
44        return u;
45    }
46
47    void build() {
48        queue<int> q;
49        for (int c = 0; c < ALPHABET; ++c) {
50            if (t[0].nxt[c]) {
51                q.push(t[0].nxt[c]);
52            }
53        }
54
55        while (!q.empty()) {
56            int u = q.front();
57            q.pop();
58            bfs_order.push_back(u);
59
60            for (int c = 0; c < ALPHABET; ++c) {
61                int v = t[u].nxt[c];
62                if (v) {
63                    t[v].link = t[t[u].link].nxt[c];
64
65                    // Compute the exit link:
66                    // If the immediate suffix link is a terminal, point to it.
67                    // Otherwise, copy its exit link.

```

```

68         t[v].exit_link = (t[t[v].link].terminal_idx != -1)
69             ? t[v].link : t[t[v].link].exit_link;
70
71         q.push(v);
72     }
73     else {
74         t[u].nxt[c] = t[t[u].link].nxt[c];
75     }
76 }
77
78 int advance(int u, char c) {
79     return t[u].nxt[get_idx(c)];
80 }
81
82 };

```

7.4 Suffix Structures

7.4.1 SuffixArrayLCP

```

1 struct SuffixArray {
2     string S;
3     // sa is the suffix array with the empty suffix being sa[0]
4     // lcp[i] holds the lcp between sa[i], sa[i - 1]
5     vector<int> logs, sa, lcp, rank;
6     vector<vector<int>> table;
7
8     SuffixArray() {};
9
10    SuffixArray(string &s, int lim = 256) {
11        S = s;
12        int n = s.size() + 1, k = 0, a, b;
13        vector<int> c(s.begin(), s.end() + 1), tmp(n), frq(max(n, lim))
14        ;
15        c.back() = 0; //0 is less than any character
16        sa = lcp = rank = tmp, iota(sa.begin(), sa.end(), 0);
17        for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p) {
18            p = j, iota(tmp.begin(), tmp.end(), n - j);
19            for (int i = 0; i < n; i++) {
20                if (sa[i] >= j)
21                    tmp[p++] = sa[i] - j;
22            }
23
24            fill(frq.begin(), frq.end(), 0);
25
26            for (int i = 0; i < n; i++) frq[c[i]]++;
27            for (int i = 1; i < lim; i++) frq[i] += frq[i - 1];
28            for (int i = n; i--;) sa[--frq[c[tmp[i]]]] = tmp[i];
29
30            swap(c, tmp), p = 1, c[sa[0]] = 0;
31            for (int i = 1; i < n; i++)
32                a = sa[i - 1], b = sa[i], c[b] = (tmp[a] == tmp[b] &&
33                    tmp[a + j] == tmp[b + j]) ? p - 1 : p++;
34
35            for (int i = 1; i < n; i++) rank[sa[i]] = i;
36            for (int i = 0, j; i < n - 1; lcp[rank[i++]] = k)
37                for (k &&k--, j = sa[rank[i] - 1];
38                    s[i + k] == s[j + k];
39                    k++);
40
41            void preLcp() {
42                int n = S.size() + 1;
43                logs = vector<int>(n + 5);
44                for (int i = 2; i < n + 5; ++i) {
45                    logs[i] = logs[i / 2] + 1;
46                }
47                table = vector<vector<int>>(n, vector<int>(20));
48                for (int i = 0; i < n; ++i) {
49                    table[i][0] = lcp[i];
50                }
51
52                for (int j = 1; j <= logs[n]; ++j) {
53                    for (int i = 0; i <= n - (1 << j); ++i) {
54                        table[i][j] = min(table[i][j - 1], table[i + (1 << (j - 1))][j - 1]);
55                    }
56                }
57            }
58
59            int queryLcp(int i, int j) {
60                // if (i == j) return (int) S.size() - i;
61                // i = rank[i], j = rank[j];
62                if (i == j) return (int) S.size() - sa[i];
63                if (i > j)
64                    swap(i, j);
65                ++i;
66                int len = logs[j - i + 1];
67                return min(table[i][len], table[j - (1 << len) + 1][len]);
68            }
69        };

```

7.4.2 SuffixAutomaton

```

1 struct SuffixAutomaton {
2     struct Node {
3         int len = 0, link = -1;
4         map<char, int> next;
5     };
6
7     vector<Node> st;
8     int sz = 1, last = 0;
9
10    SuffixAutomaton(const string& s) {
11        st.resize(s.length() * 2);
12        st[0].len = 0;
13        st[0].link = -1;
14        for (char c : s) extend(c);
15    }
16
17    void extend(char c) {
18        int cur = sz++;

```

```

19        st[cur].len = st[last].len + 1;
20        int p = last;
21        while (p != -1 && !st[p].next.count(c)) {
22            st[p].next[c] = cur;
23            p = st[p].link;
24        }
25        if (p == -1) {
26            st[cur].link = 0;
27        } else {
28            int q = st[p].next[c];
29            if (st[p].len + 1 == st[q].len) {
30                st[cur].link = q;
31            } else {
32                int clone = sz++;
33                st[clone].len = st[p].len + 1;
34                st[clone].next = st[q].next;
35                st[clone].link = st[q].link;
36                while (p != -1 && st[p].next[c] == q) {
37                    st[p].next[c] = clone;
38                    p = st[p].link;
39                }
40                st[q].link = st[cur].link = clone;
41            }
42            last = cur;
43        }
44    }
45 };

```

7.5 Palindromes

7.5.1 Manacher

```

1 struct Manacher {
2     vector<int> p;
3
4     // Returns a vector where p[i] is the radius of the longest
5     // palindrome around center i
6     // Centers are interleaved: 0 is before string, 1 is first char, 2
7     // is between 1st & 2nd, etc.
8     Manacher(string s) {
9         string t = "#";
10        for (char c : s) {
11            t += c;
12            t += "#";
13        }
14        int n = t.size();
15        p.assign(n, 0);
16        int c = 0, r = 0;
17        for (int i = 0; i < n; i++) {
18            int mirror = 2 * c - i;
19            if (i < r) p[i] = min(r - i, p[mirror]);
20            while (i - 1 - p[i] >= 0 && i + 1 + p[i] < n && t[i - 1 - p[i]] == t[i + 1 + p[i]]) {
21                p[i]++;
22            }
23            if (i + p[i] > r) {
24                c = i;
25                r = i + p[i];
26            }
27        }
28
29        // Check if substring [l, r] (0-based) is a palindrome in O(1)
30        bool is_palindrome(int l, int r) {
31            int len = r - l + 1;
32            int center = l + r + 1;
33            return p[center] >= len;
34        }
35    };

```

8 Dynamic Programming (DP)

8.1 DP Optimizations

8.1.1 DivideAndConquerDP

```

1 int a[N];
2 ll curCost;
3 int curL = 0, curR = -1, freq[N];
4 pair<int, ll> pre[N], cur[N];
5 // Modify at will :
6 int lo(int ind) { return 1; }
7 int hi(int ind) { return ind; }
8
9 void add(int val) {
10    curCost += freq[val]++;
11 }
12
13 void remove(int val) {
14    curCost -= --freq[val];
15 }
16
17 ll Cost(int l, int r) {
18    while (curR < r)
19        add(a[++curR]);
20    while (curL > l)
21        add(a[--curL]);
22    while (curR > r)
23        remove(a[curR--]);
24    while (curL < l)
25        remove(a[curL++]);
26    return curCost;
27 }
28
29 ll f(int ind, int k) { return pre[k - 1].nd + Cost(k, ind); }
30 void store(int ind, int k, ll v) { cur[ind] = pair(k, v); }
31
32 void rec(int L, int R, int LO, int HI) {
33    if (L > R) return;
34    int mid = (L + R) >> 1;
35    pair best(llinf, LO);
36    for (int k = max(LO, lo(mid)); k <= min(HI, hi(mid)); ++k)
37        best = min(best, pair(f(mid, k), k));

```

```

38     store(mid, best.second, best.first);
39     rec(L, mid - 1, L0, best.second);
40     rec(mid + 1, R, best.second, HI);
41 }
42
43 void solve(int L, int R) { rec(L, R, -inf, inf); }

```

8.1.2 ConvexHullTrick

```

1  //To query the maximum value of the function for all x in [l, r], query
2  //    only two points: l, r because the functions are convex.
3  struct Line {
4      ll m, b;
5      mutable function<const Line *(>> succ;
6
7      bool operator<(const Line &other) const {
8          return m < other.m;
9      }
10
11     bool operator<(const ll &x) const {
12         const Line *s = succ();
13         if (!s)
14             return 0;
15         return b - s->b < (s->m - m) * x;
16     };
17
18     // will maintain upper hull for maximum
19     struct HullDynamic : public multiset<Line, less<>> {
20         bool bad(iterator y) {
21             auto z = next(y);
22             if (y == begin()) {
23                 if (z == end())
24                     return 0;
25                 return y->m == z->m && y->b <= z->b;
26             }
27             auto x = prev(y);
28             if (z == end())
29                 return y->m == x->m && y->b <= x->b;
30             return (ld) (x->b - y->b) * (z->m - y->m) >= (ld) (y->b - z->b)
31                 * (y->m - x->m);
32         }
33
34         void insert_line(ll m, ll b) {
35             // for minimum
36             m *= -1;
37             // b *= -1;
38             auto y = insert({m, b});
39             y->succ = [=] { return next(y) == end() ? 0 : &*next(y); };
40             if (bad(y)) {
41                 erase(y);
42                 return;
43             }
44             while (next(y) != end() && bad(next(y)))
45                 erase(next(y));
46             while (y != begin() && bad(prev(y)))
47                 erase(prev(y));
48         }
49
50         ll query(ll x) {
51             auto l = *lower_bound(x);
52             // for minimum
53             // return -(l.m * x + l.b);
54             return l.m * x + l.b;
55         };
56     };

```

8.1.3 ConvexHullTrickRollback

```

1  // Template parameters:
2  // IS_MAX: Set to true for Maximum queries, false for Minimum queries.
3  // IS_INC: Set to true if slopes added are strictly Increasing, false
4  // for Decreasing.
5  template<bool IS_MAX = false, bool IS_INC = false>
6  struct RollbackCHT {
7      struct Line {
8          ll m, c;
9          Line() : m(0), c(llinf) {}
10         Line(ll m, ll c) : m(m), c(c) {}
11         ll eval(ll x) const { return m * x + c; }
12     };
13
14     struct History {
15         int pos;
16         int sz;
17         Line old_line;
18     };
19
20     vector<Line> st;
21     vector<History> hist;
22     int sz;
23
24     RollbackCHT(int max_nodes) {
25         st.resize(max_nodes);
26         sz = 0;
27     }
28
29     bool redundant(Line l1, Line l2, Line l3) {
30         __int128 dx1 = l1.m - l2.m;
31         __int128 dc1 = l2.c - l1.c;
32         __int128 dx2 = l2.m - l3.m;
33         __int128 dc2 = l3.c - l2.c;
34         return dc1 * dx2 >= dc2 * dx1;
35     }
36
37     void add(ll m, ll c) {
38         // The magic happens here: automatically formats internal state
39         ll m_int = IS_INC ? -m : m;
40         ll c_int = IS_MAX ? -c : c;
41
42         Line l_new(m_int, c_int);
43         int pos = sz;
44         int low = 1, high = sz - 1;

```

```

45     while (low <= high) {
46         int mid = low + (high - low) / 2;
47         if (redundant(st[mid - 1], st[mid], l_new)) {
48             pos = mid;
49             high = mid - 1;
50         } else {
51             low = mid + 1;
52         }
53     }
54
55     hist.push_back({pos, sz, st[pos]});
56     st[pos] = l_new;
57     sz = pos + 1;
58 }
59
60 void rollback() {
61     if (hist.empty()) return;
62     History h = hist.back();
63     hist.pop_back();
64     sz = h.sz;
65     st[h.pos] = h.old_line;
66 }
67
68 ll query(ll x) {
69     if (sz == 0) return IS_MAX ? -llinf : llinf;
70
71     // The magic happens here: matches x to the internal geometry
72     ll x_int = (IS_INC != IS_MAX) ? -x : x;
73
74     int low = 0, high = sz - 2;
75     int best = sz - 1;
76
77     while (low <= high) {
78         int mid = low + (high - low) / 2;
79         if (st[mid].eval(x_int) <= st[mid + 1].eval(x_int)) {
80             best = mid;
81             high = mid - 1;
82         } else {
83             low = mid + 1;
84         }
85     }
86
87     ll ans = st[best].eval(x_int);
88     return IS_MAX ? -ans : ans;
89 }
90 };

```

8.1.4 LiChaoTree

```

1  // To get an instance : node *root = EMPTY;
2  typedef pair<long long, long long> line;
3  int start_x, end_x;
4  line neutral{0, -1e18};
5
6  ll sub(const line &l, ll x) {
7      return l.first * x + l.second;
8  }
9
10 double inter(const line &l1, const line &l2) {
11     return (l2.second - l1.second) / (l1.first - l2.first - .0);
12 }
13
14 extern struct node *const EMPTY;
15 struct node {
16     line l;
17     node *lf, *rt;
18     node() : l(neutral), lf(this), rt(this) {}
19     node(line l) : l(l), lf(EMPTY), rt(EMPTY) {}
20 };
21 node *const EMPTY = new node;
22
23 void insert(line l, int lx, int rx, node *&cur, ll ns = start_x, ll ne = end_x) {
24     if (rx < ns || lx > ne) return;
25     if (ns >= lx && ne <= rx) {
26         if (cur == EMPTY) {
27             cur = new node(l);
28             return;
29         }
30         if (l.first == cur->l.first) {
31             cur->l = max(cur->l, l);
32             return;
33         }
34
35         auto x = inter(l, cur->l);
36         if (x < ns || x > ne) {
37             if (sub(l, ns) > sub(cur->l, ns)) cur->l = l;
38             return;
39         }
40         ll mid = ns + (ne - ns) / 2;
41         if (x < mid) {
42             if (sub(l, ne) > sub(cur->l, ne)) swap(l, cur->l);
43             insert(l, lx, rx, cur->lf, ns, mid);
44         } else {
45             if (sub(l, ns) > sub(cur->l, ns)) swap(l, cur->l);
46             insert(l, lx, rx, cur->rt, mid + 1, ne);
47         }
48         return;
49     }
50
51     if (cur == EMPTY) {
52         cur = new node(neutral);
53     }
54     ll mid = ns + (ne - ns) / 2;
55     insert(l, lx, rx, cur->lf, ns, mid);
56     insert(l, lx, rx, cur->rt, mid + 1, ne);
57 }
58
59 ll query(ll x, node *cur, ll ns = start_x, ll ne = end_x) {
60     auto ret = sub(cur->l, x);
61     if (x < ns || x > ne)
62         return sub(neutral, x);
63     if (ns == ne)
64         return ret;

```

```

65     ll mid = ns + (ne - ns) / 2;
66     if (x <= mid)
67         return max(ret, query(x, cur->l, ns, mid));
68     return max(ret, query(x, cur->r, mid + 1, ne));
69 }

```

8.1.5 PersistentLiChaoTree

```

1  const ll maxN = 1e7 + 5;
2
3  struct Line {
4      ll m, c;
5
6      Line() : m(0), c(1llinf) {}
7
8      Line(ll m, ll c) : m(m), c(c) {}
9  };
10
11 ll sub(ll x, Line l) {
12     return x * l.m + l.c;
13 }
14
15 // Persistent Li Chao
16 struct Node {
17     // range I am responsible for
18     Line line;
19     Node *left, *right;
20
21     Node() {
22         left = right = NULL;
23     }
24
25     Node(ll m, ll c) {
26         line = Line(m, c);
27         left = right = NULL;
28     }
29
30     void extend(int l, int r) {
31         if (left == NULL && l != r) {
32             left = new Node();
33             right = new Node();
34         }
35     }
36
37     Node *copy(Node *node) {
38         Node *newNode = new Node;
39         newNode->left = node->left;
40         newNode->right = node->right;
41         newNode->line = node->line;
42         return newNode;
43     }
44
45     Node *add(Line toAdd, int l, int r) {
46         int mid = (l + r) / 2;
47         Node *cur = copy(this);
48         if (l == r) {
49             if (sub(l, toAdd) < sub(l, cur->line))
50                 swap(toAdd, cur->line);
51             return cur;
52         }
53         bool lef = sub(l, toAdd) < sub(l, cur->line);
54         bool midE = sub(mid + 1, toAdd) < sub(mid + 1, cur->line);
55         if (midE)
56             swap(cur->line, toAdd);
57         cur->extend(l, r);
58         if (lef != midE) {
59             cur->left = cur->left->add(toAdd, l, mid);
60         } else {
61             if (mid + 1 > r)
62                 return cur;
63             cur->right = cur->right->add(toAdd, mid + 1, r);
64         }
65         return cur;
66     }
67
68     Node *add(Line toAdd) {
69         return add(toAdd, -maxN + 1, maxN - 1);
70     }
71
72     ll query(ll x, int l, int r) {
73         int mid = (l + r) / 2;
74         if (l == r || left == NULL)
75             return sub(x, line);
76         extend(l, r);
77         if (x <= mid)
78             return min(sub(x, line), left->query(x, l, mid));
79         else
80             return min(sub(x, line), right->query(x, mid + 1, r));
81     }
82
83     ll query(ll x) {
84         return query(x, -maxN + 1, maxN - 1);
85     }
86
87     void clear() {
88         if (left != NULL) {
89             left->clear();
90             right->clear();
91         }
92         delete this;
93     }
94 };
95
96 Node *tree[N];

```

8.2 Bitmask DP

8.2.1 SOS_DP

```

1 // Computes the sum of all subsets for every bitmask in O(N * 2^N)
2 vector<long long> SOS_DP(int n_bits, const vector<long long>& a) {
3     int max_mask = 1 << n_bits;
4     vector<long long> dp = a; // dp[mask] initially holds exactly the
    value for mask

```

```

5
6     for (int i = 0; i < n_bits; ++i) {
7         for (int mask = 0; mask < max_mask; ++mask) {
8             if (mask & (1 << i)) {
9                 dp[mask] += dp[mask ^ (1 << i)];
10            }
11        }
12    }
13    return dp; // dp[mask] now contains sum of all submasks of mask
14 }

```

9 Trees

9.1 Lowest Common Ancestor (LCA)

9.1.1 BinaryLiftingLCA

```

1 struct LCA {
2     static const int B = 21;
3     vector<array<int, 2>> tour;
4     vector<array<array<int, 2>, B>> T;
5     vector<int> d, in, lg;
6
7     LCA(int n, auto &adj): d(n + 1), in(n + 1), lg(n*2 + 1) {
8         tour.reserve(n * 2);
9         T.resize(n);
10        dfs(0, 0, adj);
11        for (int i = 0; i < n; ++i) lg[i] = __lg(i), T[i][0] = tour[i];
12        for (int j = 1; j < B; ++j)
13            for (int i = 0; i + (1 << j) - 1 < n; ++i)
14                T[i][j] = min(T[i][j - 1], T[i + (1 << j) - 1][j - 1]);
15    }
16
17    void dfs(int u, int p, auto &adj) {
18        in[u] = size(tour);
19        tour.push_back({d[u], u});
20        for (int v: adj[u]) {
21            if (v == p) continue;
22            d[v] = d[u] + 1;
23            dfs(v, u, adj);
24            tour.push_back({d[u], u});
25        }
26    }
27
28    int get(int u, int v) {
29        int l = min(in[u], in[v]), r = max(in[u], in[v]);
30        int j = lg[r - l + 1];
31        return min(T[l][j], T[r - (1 << j) + 1][j])[1];
32    }
33
34    int dist(int u, int v) { return d[u] + d[v] - 2 * d[get(u, v)]; }
35 };

```

9.2 DSU on Tree

9.2.1 DSUOnTree_Sack

```

1 class DSUOnTree {
2 private:
3     struct Query {
4         int idx, x;
5     };
6
7     int n;
8     vector<vector<int>> adj;
9     vector<int> sz, depth;
10    vector<bool> big;
11    vector<vector<Query>> queries;
12    vector<int> ans;
13
14    // =====
15    // PROBLEM SPECIFIC STATE VARIABLES
16    // =====
17    vector<char> c; // Example: Array of characters per node
18
19    // Example state variables
20    // int distinct_count = 0;
21    // vector<int> freq;
22
23    void add(int u, int p) {
24        // --- ADD LOGIC HERE ---
25        // Example: freq[c[u] - 'a']++;
26
27        for (int v: adj[u]) {
28            if (v == p || big[v]) continue; // Skip parent and heavy
29                child[cite: 4]
30            add(v, u);
31        }
32    }
33
34    void remove(int u, int p) {
35        // --- REMOVE LOGIC HERE ---
36        // Example: freq[c[u] - 'a']--;
37
38        for (int v: adj[u]) {
39            if (v == p || big[v]) continue; // Skip parent and heavy
40                child[cite: 4]
41            remove(v, u);
42        }
43    }
44
45    int get_answer(int u, int x = 0) {
46        // --- QUERY ANSWER LOGIC HERE ---
47        return 0;
48    }
49
50    void pre(int u, int p = 0) {
51        sz[u] = 1;
52        depth[u] = depth[p] + 1;
53        for (int v: adj[u]) {
54            if (v == p) continue;
55            pre(v, u);

```

```

55         sz[u] += sz[v]; // Compute subtree size[cite: 4]
56     }
57 }
58
59 void dfs(int u, int p = 0, bool keep = false) {
60     int mx = -1, bigChild = -1;
61
62     // Find the heavy child[cite: 4]
63     for (int v : adj[u]) {
64         if (v == p) continue;
65         if (sz[v] > mx) {
66             mx = sz[v];
67             bigChild = v;
68         }
69     }
70
71     // Process light children and clear them[cite: 4]
72     for (int v : adj[u]) {
73         if (v == p || v == bigChild) continue;
74         dfs(v, u, false);
75     }
76
77     // Process heavy child and keep it[cite: 4]
78     if (bigChild != -1) {
79         dfs(bigChild, u, true);
80         big[bigChild] = true;
81     }
82
83     // Add current node and its light children[cite: 4]
84     add(u, p);
85
86     // Answer all queries for the current node[cite: 4]
87     for (const auto& q : queries[u]) {
88         ans[q.idx] = get_answer(u, q.x);
89     }
90
91     // Reset the big child flag[cite: 4]
92     if (bigChild != -1) {
93         big[bigChild] = false;
94     }
95
96     // Clear state if this node is not meant to be kept[cite: 4]
97     if (!keep) {
98         remove(u, p);
99     }
100 }
101
102 public:
103     // Initialize with 1-based indexing in mind
104     DSUOnTree(int nodes, const vector<char>& chars) : n(nodes), c(chars)
105     {
106         adj.assign(n + 1, vector<int>());
107         sz.assign(n + 1, 0);
108         depth.assign(n + 1, 0);
109         big.assign(n + 1, false);
110         queries.assign(n + 1, vector<Query>());
111         // Initialize problem-specific arrays here (e.g., freq.assign
112         // (26, 0);)
113     }
114
115     void add_edge(int u, int v) {
116         adj[u].push_back(v);
117         adj[v].push_back(u);
118     }
119
120     void add_query(int node, int query_idx, int x = 0) {
121         queries[node].push_back({query_idx, x});
122     }
123
124     vector<int> solve(int root, int num_queries) {
125         ans.assign(num_queries, 0);
126         pre(root, 0); // Precalculate sizes and depths[cite: 4]
127         dfs(root, 0, false); // Run the sack algorithm[cite: 4]
128         return ans;
129     }
130 };

```

9.3 Heavy Light Decomposition (HLD)

9.3.1 HeavyLightDecomposition

```

1 class HLD {
2 public:
3     vector<int> par, sz, head, tin, tout, who, depth;
4
5     int dfs1(int u, vector<vector<int>> &adj) {
6         for (int &v: adj[u]) {
7             if (v == par[u]) continue;
8             depth[v] = depth[u] + 1;
9             par[v] = u;
10            sz[u] += dfs1(v, adj);
11            if (sz[v] > sz[adj[u][0]] || adj[u][0] == par[u]) swap(v,
12                adj[u][0]);
13        }
14        return sz[u];
15    }
16
17    void dfs2(int u, int &timer, const vector<vector<int>> &adj) {
18        tin[u] = timer++;
19        for (int v: adj[u]) {
20            if (v == par[u]) continue;
21            head[v] = (timer == tin[u] + 1 ? head[u] : v);
22            dfs2(v, timer, adj);
23        }
24        tout[u] = timer - 1;
25    }
26
27    HLD(vector<vector<int>> adj, int r = 0)
28        : par(adj.size(), -1), sz(adj.size(), 1), head(adj.size(), r),
29        tin(adj.size()), tout(adj.size()),
30        who(adj.size()),
31        depth(adj.size()) {
32        dfs1(r, adj);
33        int x = 0;

```

```

32        dfs2(r, x, adj);
33        for (int i = 0; i < adj.size(); ++i) who[tin[i]] = i;
34    }
35
36    vector<pair<int, int>> path(int u, int v) {
37        vector<pair<int, int>> res;
38        for (; v = par[head[v]]; ) {
39            if (depth[head[u]] > depth[head[v]]) swap(u, v);
40            if (head[u] != head[v]) {
41                res.emplace_back(tin[head[v]], tin[v]);
42            } else {
43                if (depth[u] > depth[v]) swap(u, v);
44                res.emplace_back(tin[u], tin[v]);
45                return res;
46            }
47        }
48    }
49
50    pair<int, int> subtree(int u) {
51        return {tin[u], tout[u]};
52    }
53
54    int dist(int u, int v) {
55        return depth[u] + depth[v] - 2 * depth[lca(u, v)];
56    }
57
58    int lca(int u, int v) {
59        for (; v = par[head[v]]; ) {
60            if (depth[head[u]] > depth[head[v]]) swap(u, v);
61            if (head[u] == head[v]) {
62                if (depth[u] > depth[v]) swap(u, v);
63                return u;
64            }
65        }
66    }
67
68    bool isAncestor(int u, int v) {
69        return tin[u] <= tin[v] && tout[u] >= tout[v];
70    }
71 };

```

9.4 Centroid Decomposition

9.4.1 CentroidDecomposition

```

1 class CentroidDecomposition {
2 private:
3     int n;
4     vector<vector<int>> adj;
5     vector<int> sz, par, nodes;
6     vector<bool> vis;
7
8     int dfsSz(int u, int p) {
9         sz[u] = 1;
10        for (int v : adj[u]) {
11            if (vis[v] || v == p) continue;
12            sz[u] += dfsSz(v, u);
13        }
14        return sz[u];
15    }
16
17    int getCentroid(int u, int p, int total_nodes) {
18        for (int v : adj[u]) {
19            if (vis[v] || v == p) continue;
20            if (sz[v] * 2 > total_nodes) {
21                return getCentroid(v, u, total_nodes);
22            }
23        }
24        return u;
25    }
26
27    void collect(int u, int p) {
28        nodes.push_back(u);
29        for (int v : adj[u]) {
30            if (vis[v] || v == p) continue;
31            collect(v, u);
32        }
33    }
34
35    void build(int u, int p) {
36        int total_nodes = dfsSz(u, p);
37        int centroid = getCentroid(u, p, total_nodes);
38
39        if (p != -1) {
40            par[centroid] = p;
41        } else {
42            par[centroid] = centroid;
43        }
44
45        vis[centroid] = true;
46
47        // =====
48        // PROBLEM SPECIFIC PROCESSING
49        // =====
50        // Process paths going through the 'centroid' here
51        for (int v : adj[centroid]) {
52            if (vis[v]) continue;
53            nodes.clear();
54            collect(v, centroid);
55
56            // e.g., Update answers using the nodes collected from this
57            // subtree branch
58        }
59        // =====
60
61        // Recursively decompose the remaining components
62        for (int v : adj[centroid]) {
63            if (vis[v]) continue;
64            build(v, centroid);
65        }
66    }
67 public:
68     // Initialize with 1-based indexing

```



```

69 CentroidDecomposition(int nodes_count) : n(nodes_count) {
70     adj.assign(n + 1, vector<int>());
71     sz.assign(n + 1, 0);
72     par.assign(n + 1, 0);
73     vis.assign(n + 1, false);
74 }
75
76 void add_edge(int u, int v) {
77     adj[u].push_back(v);
78     adj[v].push_back(u);
79 }
80
81 // Start decomposition, usually from node 1
82 void decompose(int root = 1) {
83     build(root, -1);
84 }
85
86 // Returns the parent of a node in the centroid tree
87 int get_centroid_parent(int u) const {
88     return par[u];
89 }
90 };

```

10 Game Theory

11 Geometry

11.1 Geometry 2D

11.1.1 Point2D

```

1  template<class T>
2  int sgn(T x) { return (x > 0) - (x < 0); }
3
4  template<class T>
5  struct Point {
6      typedef Point P;
7      T x, y;
8
9      Point(T x = 0, T y = 0) : x(x), y(y) {}
10
11      bool operator<(P p) const { return tie(x, y) < tie(p.x, p.y); }
12
13      bool operator==(P p) const { return tie(x, y) == tie(p.x, p.y); }
14
15      P operator+(P p) const { return P(x + p.x, y + p.y); }
16
17      P operator-(P p) const { return P(x - p.x, y - p.y); }
18
19      P operator*(T d) const { return P(x * d, y * d); }
20
21      P operator/(T d) const { return P(x / d, y / d); }
22
23      T dot(P p) const { return x * p.x + y * p.y; }
24
25      T cross(P p) const { return x * p.y - y * p.x; }
26
27      T cross(P a, P b) const { return (a - *this).cross(b - *this); }
28
29      T dist2() const { return x * x + y * y; }
30
31      T dist2(P b) const { return (*this - b).dist2(); }
32
33      double dist() const { return sqrt((double) dist2()); }
34
35      double dist(P b) { return (*this - b).dist(); }
36
37      // angle to x-axis in interval [-pi, pi]
38      double angle() const { return atan2(y, x); }
39
40      P unit() const { return *this / dist(); } // makes dist()==1
41      P perp() const { return P(-y, x); } // rotates +90 degrees
42      P normal() const { return perp().unit(); }
43
44      P getVector(const P &p) const { return p - (*this); }
45
46      // returns point rotated 'a' radians ccw around the origin
47      P rotate(double a) const {
48          return P(x * cos(a) - y * sin(a), x * sin(a) + y * cos(a));
49      }
50
51      friend ostream &operator<<(ostream &os, P p) {
52          // return os << "(" << p.x << ", " << p.y << ")";
53          return os << p.x << " " << p.y;
54      }
55
56      friend istream &operator>>(istream &is, P &p) {
57          return is >> p.x >> p.y;
58      }
59
60
61      // Project point onto line through a and b (assuming a != b).
62      P projectOnLine(const P &a, const P &b) const {
63          P ab = a.getVector(b);
64          P ac = a.getVector(*this);
65          return a + ab * ac.dot(ab) / a.dist2(b);
66      }
67
68      // Project point c onto line segment through a and b (assuming a != b).
69      P projectOnSegment(const P &a, const P &b) const {
70          P &c = (P &) *this;
71          P ab = a.getVector(b);
72          P ac = a.getVector(c);
73
74          long double r = ac.dot(ab), d = a.dist2(b);
75          if (r < 0) return a;
76          if (r > d) return b;
77
78          return a + ab * r / d;
79      }
80
81      P reflectAroundLine(const P &a, const P &b) const {
82          return projectOnLine(a, b) * 2 - (*this);

```

```

83     }
84 };
85
86 int dcmp(const ld &a, const ld &b){
87     if(fabs(a - b) < eps)
88         return 0;
89
90     return (a > b ? 1 : -1);
91 }

```

11.1.2 Line2D

```

1  template<class T>
2  double lineDist(T p, T s, T e) {
3      if (s == e) {
4          return s.dist(p);
5      }
6      return fabs((p - s).cross(e - s) / (e - s).dist());
7  }
8
9  template<class T>
10 pair<int, T> lineInter(T s1, T e1, T s2, T e2) {
11     // first = 0 no intersection
12     // first = 1 intersection
13     // first = -1 infinite intersection
14     auto d = (e1 - s1).cross(e2 - s2);
15     if (dcmp(d, 0) == 0) // if parallel same line first = -1 else
16         first = 0
17         return {-(s1.cross(e1, s2) == 0), {0, 0}};
18     auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
19     return {1, (s1 * p + e1 * q) / d};
20 }
21
22
23 template<class T>
24 bool onSegment(T p, T s, T e) {
25     return dcmp(p.cross(s, e), 0) == 0 && dcmp((s - p).dot(e - p), 0)
26         <= 0;
27 }
28
29 template<class T>
30 double segDist(T p, T s, T e) {
31     if (dcmp((p - s).dot(e - s), 0) <= 0) return s.dist(p);
32     if (dcmp((p - e).dot(e - s), 0) >= 0) return e.dist(p);
33     return lineDist(p, s, e);
34 }
35
36 template<class T>
37 pair<int, T> segInter(T s1, T e1, T s2, T e2) {
38     // first = 0 no intersection
39     // first = 1 intersection
40     // first = -1 infinite intersection
41     pair<int, T> ret = lineInter(s1, e1, s2, e2);
42     if (ret.first == 0) return ret;
43     else if (ret.first == 1) {
44         if (onSegment(ret.second, s1, e1) && onSegment(ret.second, s2,
45             e2))
46             return ret;
47         else
48             return {0, {0, 0}};
49     } else {
50         if (onSegment(s1, s2, e2) || onSegment(e1, s2, e2))
51             return {-1, (onSegment(s1, s2, e2) ? s1 : e1)};
52         else if (onSegment(s2, s1, e1) || onSegment(e2, s1, e1))
53             return {-1, (onSegment(s2, s1, e1) ? s2 : e2)};
54         else
55             return {0, {0, 0}};
56     }
57 }
58
59 template<class T>
60 T closestOnSegment(T p, T s, T e) {
61     if ((p - s).dot(e - s) <= 0) return s;
62     else if ((p - e).dot(e - s) >= 0) return e;
63     else return p.projectOnLine(s, e);
64 }
65
66 template<class T>
67 double segSegDist(T s1, T e1, T s2, T e2) {
68     if (segInter(s1, e1, s2, e2).first != 0)
69         return 0;
70     double ret = min({segDist(s1, s2, e2), segDist(e1, s2, e2), segDist(s2,
71         s1, e1), segDist(e2, s1, e1)});
72     return ret;
73 }
74
75
76 template<class T>
77 bool onRay(T p, T s, T e) {
78     return dcmp(p.cross(s, e), 0) == 0 && dcmp((p - s).dot(e - s), 0)
79         >= 0;
80 }
81
82 template<class T>
83 double rayDist(T p, T s, T e) {
84     if ((p - s).dot(e - s) <= 0) {
85         return s.dist(p);
86     }
87     return lineDist(p, s, e);
88 }
89
90 template<class T>
91 pair<int, T> rayInter(T s1, T e1, T s2, T e2) {
92     // first = 0 no intersection
93     // first = 1 intersection
94     // first = -1 infinite intersection
95     pair<int, T> ret = lineInter(s1, e1, s2, e2);
96     if (ret.first == 0) return ret;
97     else if (ret.first == 1) {

```

```
98         if (onRay(ret.second, s1, e1) && onRay(ret.second, s2, e2))
99             return ret;
100     else
101         return {0, {0,0}};
102 } else {
103     if (onRay(s1, s2, e2) || onRay(s2, s1, e1))
104         return {-1, onRay(s1, s2, e2) ? s1:s2};
105     else
106         return {0, {0,0}};
107 }
108 }
109
110 template<class T>
111 double rayRayDist(T s1, T e1, T s2, T e2) {
112     if (rayInter(s1, e1, s2, e2).first != 0)
113         return 0;
114     double ret = min(rayDist(s1, s2, e2), rayDist(s2, s1, e1));
115     return ret;
116 }
```

11.1.3 AngleOperations

```
1 template<class T>
2 // angle between [0, 2*pi]
3 ld angleBetween(T a, T b) {
4     ld ret = atan2(a.cross(b), a.dot(b));
5     if (dcmp(ret, 0) == -1) {
6         ret += 2 * PI;
7     }
8     // return min(ret, 2 * PI - ret); //to return the smaller angle
9     return ret;
10 }
11
12 template<class T>
13 ld angle0(T a, T O, T b) { /// angle(aOb)
14     assert(a.dist(O) > eps && b.dist(O) > eps); // nan
15     T v1 = (a - O), v2 = (b - O);
16     return angleBetween(v1, v2);
17 }
18
19
20 // [-pi, pi]
21 ld format(ld angle) {
22     while (dcmp(angle, PI) == 1)
23         angle -= 2 * PI;
24     while (dcmp(angle, -PI) != 1)
25         angle += 2 * PI;
26     return angle;
27 }
```

11.1.4 PolygonArea

```
1 template<class T>
2 ld polygonArea(vector<T>&v){
3     ld ret = 0;
4     int n = v.size();
5     for (int i = 0; i < n; ++i) {
6         ret += v[i].cross(v[(i+1)%n]);
7     }
8     return 0.5 * abs(ret);
9 }
```

11.1.5 Circle3Points

```
1 typedef Point<double> P;
2
3 bool isColliner(const P &A, const P &B, const P &C)
4 {
5     return dcmp(P(B - A).cross(P(C - A)), 0) == 0;
6 }
7
8 P ccCenter(const P &A, const P &B, const P &C)
9 {
10     P b = C - A, c = B - A;
11     return A + (b * c.dist2() - c * b.dist2()).perp() / b.cross(c) / 2;
12 }
13
14 double ccRadius(const P &A, const P &B, const P &C)
15 {
16     return (B - A).dist() * (C - B).dist() * (A - C).dist() /
17         abs((B - A).cross(C - A)) / 2;
18 }
```

11.1.6 CircleLineIntersection

```
1 template<class T>
2 vector<T> circleLine(T o, double r, T a, T b) {
3     double h2 = r * r - lineDist(o, a, b) * lineDist(o, a, b);
4     if (dcmp(h2, 0) != -1) { // the line touches the circle
5         T p = o.projectOnLine(a, b); // point P
6         T h = (a - b).unit() * sqrt(h2); // vector parallel to l, of
7             // length h
8         if (p - h == p + h)
9             return {p - h};
10         else
11             return {p - h, p + h};
12     }
13     return {};
14 }
15
16 template<class T>
17 vector<T> circleSegment(T o, double r, T a, T b) {
18     vector<T> v = circleLine(o, r, a, b);
19     vector<T> out;
20     for (auto x: v)
21         if (onSegment(x, a, b))
22             out.pb(x);
23     return out;
24 }
```

11.1.7 CircleCircleIntersection

```
1 template<class T>
2 ld circleInter(T a, T b, ld r1, ld r2) {
3     if (r1 > r2) {
4         swap(r1, r2);
5         swap(a, b);
6     }
7     ld d = sqrt((a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y));
8     if (d >= r1 + r2) {
9         return 0;
10     }
11     if (d + r1 <= r2) {
12         return PI * r1 * r1;
13     }
14     ld theta = acos((d * d + r2*r2 - r1*r1)/(2 * d * r2));
15     ld alpha = acos((d * d + r1*r1 - r2*r2)/(2 * d * r1));
16     ld s1 = theta * r2 * r2;
17     ld t1 = 0.5 * r2 * r2 * sin(2 * theta);
18     ld s2 = alpha * r1 * r1;
19     ld t2 = 0.5 * r1 * r1 * sin(2 * alpha);
20     return (s1 - t1) + (s2 - t2);
21 }
```

11.1.8 ConvexHullAndrew

```
1 template<class T>
2 vector<T> convexHull(vector<T> pts, bool inc_collinear = false) {
3
4     T p0 = *min_element(pts.begin(), pts.end(), [](T &a, T &b) {
5         return make_pair(a.y, a.x) < make_pair(b.y, b.x);
6     });
7
8     sort(pts.begin(), pts.end(), [&p0](T &a, T &b) {
9         ll o = p0.cross(a, b);
10        if (o != 0) return o > 0;
11        return (a - p0).dist2() < (b - p0).dist2();
12    });
13
14    if (inc_collinear) {
15        int ind = pts.size() - 1;
16        while (ind >= 0 && p0.cross(pts[ind], pts.back()) == 0) --ind;
17        reverse(pts.begin() + ind + 1, pts.end());
18    }
19
20    vector<T> ch;
21    for (int i = 0; i < (int) pts.size(); i++) {
22        int sz = ch.size();
23        while (ch.size() > 1 &&
24            (ch[sz - 2].cross(ch[sz - 1], pts[i]) < 0 ||
25            (!inc_collinear && ch[sz - 2].cross(ch[sz - 1], pts[i])
26                == 0))) {
27            ch.pop_back();
28            sz = ch.size();
29        }
30        ch.push_back(pts[i]);
31    }
32    return ch;
33 }
```

11.1.9 RotatingCalipers

```
1 template<class T>
2 ll hullDiameter(vector<T> S) {
3     int n = S.size(), j = n < 2 ? 0 : 1;
4     ll ret = 0;
5     for (int i = 0; i < j; ++i) {
6         for (; j = (j + 1) % n) {
7             ret = max(ret, (ll)(S[i] - S[j]).dist2());
8             if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i]) >= 0)
9                 break;
10         }
11     }
12     // returns the squared diameter
13     return ret;
14 }
15
16 template<class T>
17 ld hullWidth(vector<T> S) {
18     int n = S.size();
19     if (n <= 2) return 0;
20     int i = 0, j = 1;
21     ld ret = 1e18;
22     while (i < n) {
23         while ((S[(i + 1) % n] - S[i]).cross(S[(j + 1) % n] - S[j]) >= 0) j = (j + 1) % n;
24         ret = min(ret, lineDist(S[j], S[i], S[(i+1)%n]));
25         i++;
26     }
27     return ret;
28 }
```

11.1.10 MinimumEnclosingCircle

```
1 typedef Point<double> P;
2 #define rep(aa, bb, cc) for(int aa = bb; aa < cc; aa++)
3
4 P ccCenter(const P &A, const P &B, const P &C) {
5     P b = C - A, c = B - A;
6     return A + (b * c.dist2() - c * b.dist2()).perp() / b.cross(c) / 2;
7 }
8
9 pair<P, double> mec(vector<P> ps) {
10     shuffle(all(ps), mt19937(time(0)));
11     P o = ps[0];
12     double r = 0, EPS = 1 + 1e-8;
13     rep(i, 0, ps.size()) if ((o - ps[i]).dist() > r * EPS) {
14         o = ps[i], r = 0;
15         rep(j, 0, i) if ((o - ps[j]).dist() > r * EPS) {
16             o = (ps[i] + ps[j]) / 2;
17             r = (o - ps[i]).dist();
18             rep(k, 0, j) if ((o - ps[k]).dist() > r * EPS) {
19                 o = ccCenter(ps[i], ps[j], ps[k]);
20                 r = (o - ps[i]).dist();
21             }
22         }
23     }
```

```

21     }
22     }
23 }
24     return {o, r};
25 }

```

11.1.11 HalfPlaneIntersection

```

1 // Basic half-plane struct.
2 template<class T>
3 struct Halfplane {
4
5     // 'p' is a passing point of the line and 'pq' is the direction
6     // vector of the line.
7     T p, pq;
8     long double angle;
9
10    Halfplane() {}
11    Halfplane(const T& a, const T& b) : p(a), pq(b - a) {
12        angle = atan2l(pq.y, pq.x);
13    }
14
15    // Check if point 'r' is outside this half-plane.
16    // Every half-plane allows the region to the LEFT of its line.
17    bool out(const T& r) {
18        return dcmp(pq.cross(r - p), 0) < 0;
19    }
20
21    // Comparator for sorting.
22    bool operator < (const Halfplane& e) const {
23        return angle < e.angle;
24    }
25
26    // Intersection point of the lines of two half-planes. It is
27    // assumed they're never parallel.
28    friend T inter(const Halfplane& s, const Halfplane& t) {
29        long double alpha = (ld)(t.p - s.p).cross(t.pq) / s.pq.cross(t.pq);
30        return s.p + (s.pq * alpha);
31    }
32 };
33
34 template<class T>
35 vector<T> hpIntersect(vector<Halfplane<T>> H) {
36     ld inf = 1e9;
37     T box[4] = { // Bounding box in CCW order
38         Point(inf, inf),
39         Point(-inf, inf),
40         Point(-inf, -inf),
41         Point(inf, -inf)
42     };
43
44     for(int i = 0; i < 4; i++) { // Add bounding box half-planes.
45         Halfplane aux(box[i], box[(i+1) % 4]);
46         H.push_back(aux);
47     }
48
49     // Sort by angle and start algorithm
50     sort(H.begin(), H.end());
51     deque<Halfplane<T>> dq;
52     int len = 0;
53     for(int i = 0; i < int(H.size()); i++) {
54
55         // Remove from the back of the deque while last half-plane is
56         // redundant
57         while (len > 1 && H[i].out(inter(dq[len-1], dq[len-2]))) {
58             dq.pop_back();
59             --len;
60         }
61
62         // Remove from the front of the deque while first half-plane is
63         // redundant
64         while (len > 1 && H[i].out(inter(dq[0], dq[1]))) {
65             dq.pop_front();
66             --len;
67         }
68
69         // Special case check: Parallel half-planes
70         if (len > 0 && fabs1(H[i].pq.cross(dq[len-1].pq)) < eps) {
71             // Opposite parallel half-planes that ended up checked
72             // against each other.
73             if (dcmp(H[i].pq.dot(dq[len-1].pq), 0) < 0)
74                 return vector<T>();
75
76             // Same direction half-plane: keep only the leftmost half-
77             // plane.
78             if (H[i].out(dq[len-1].p)) {
79                 dq.pop_back();
80                 --len;
81             }
82             else continue;
83         }
84
85         // Add new half-plane
86         dq.push_back(H[i]);
87         ++len;
88     }
89
90     // Final cleanup: Check half-planes at the front against the back
91     // and vice-versa
92     while (len > 2 && dq[0].out(inter(dq[len-1], dq[len-2]))) {
93         dq.pop_back();
94         --len;
95     }
96
97     while (len > 2 && dq[len-1].out(inter(dq[0], dq[1]))) {
98         dq.pop_front();
99         --len;
100    }
101
102    // Report empty intersection if necessary
103    if (len < 3) return vector<T>();
104
105    // Reconstruct the convex polygon from the remaining half-planes.

```

```

99 vector<T> ret(len);
100 for(int i = 0; i < len; i++) {
101     ret[i] = inter(dq[i], dq[(i+1)%len]);
102 }
103 return ret;
104 }

```

11.2 Geometry 3D

11.2.1 Point3D

```

1 template<class T>
2 struct Point3D {
3     typedef Point3D P;
4     typedef const P &R;
5     T x, y, z;
6
7     explicit Point3D(T x = 0, T y = 0, T z = 0) : x(x), y(y), z(z) {}
8
9     bool operator<(R p) const {
10         return tie(x, y, z) < tie(p.x, p.y, p.z);
11     }
12
13     bool operator==(R p) const {
14         return tie(x, y, z) == tie(p.x, p.y, p.z);
15     }
16
17     P operator+(R p) const { return P(x + p.x, y + p.y, z + p.z); }
18
19     P operator-(R p) const { return P(-x, -y, -z); }
20
21     P operator-(R p) const { return P(x - p.x, y - p.y, z - p.z); }
22
23     P operator*(T d) const { return P(x * d, y * d, z * d); }
24
25     P operator/(T d) const { return P(x / d, y / d, z / d); }
26
27     T dot(R p) const { return x * p.x + y * p.y + z * p.z; }
28
29     P cross(R p) const {
30         return P(y * p.z - z * p.y, z * p.x - x * p.z, x * p.y - y * p.x);
31     }
32
33     T dist2() const { return x * x + y * y + z * z; }
34
35     double dist() const { return sqrt((double) dist2()); }
36
37     //Azimuthal angle ( longitude) to x=axis in interval [pi , pi ]
38     double phi() const { return atan2(y, x); }
39
40     //Zenith angle ( latitude) to the z=axis in interval [0 , pi ]
41     double theta() const { return atan2(sqrt(x * x + y * y), z); }
42
43     P unit() const { return *this / (T) dist(); } //makes dist ()=1
44     //returns unit vector normal to *this and p
45     P normal(P p) const { return cross(p).unit(); }
46
47     //returns point rotated 'angle ' radians ccw around axis
48     P rotate(double angle, P axis) const {
49         double s = sin(angle), c = cos(angle);
50         P u = axis.unit();
51         return u * dot(u) * (1 - c) + (*this) * c - cross(u) * s;
52     }
53
54     friend ostream &operator<<(ostream &os, P p) {
55         // return os << "(" << p.x << ", " << p.y << ")";
56         return os << p.x << " " << p.y << " " << p.z;
57     }
58
59     friend istream &operator>>(istream &is, P &p) {
60         return is >> p.x >> p.y >> p.z;
61     }
62 };
63
64 namespace Axis {
65     Point3D<double> x(1, 0, 0);
66     Point3D<double> y(0, 1, 0);
67     Point3D<double> z(0, 0, 1);
68 }

```